M68FTN(D2)

MARCH 1980

M6800/M6809

MDOS FORTRAN

REFERENCE MANUAL

First Edition

TABLE OF CONTENTS Page

TABLE OF CONTENTS

CHAPTER 1
MDOS FORTRAN COMPILER

1.1 INTRODUCTION

The Motorola 6800/6809 MDOS FORTRAN compiler is designed for the solution of small to medium scale scientific problems and control applications. The system consists of computer hardware and software. There are two phases to any FORTRAN program...the COMPILATION phase and the EXECUTION or RUN-TIME phase.

For the compilation phase, the minimum configuration is:

- EXORciser or EXORterm 200 or above
- 48K bytes of memory
- EXORdisk drive with MDOS disk operating system
- ASCII terminal (may be EXORterm above)
- MDOS Editor
- MDOS Linking Loader

The minimum compile phase configuration may be expanded to include more memory, up to four disk drives, and a variety of line printer and CRT terminal devices.

This version of FORTRAN is written to support both EXORciser/EXORterm and Micromodule configurations during the execution phase of a FORTRAN program. The object code produced by the compile/link process may be stored on diskette or may be burned into PROM or EROM. Neither EXbug nor MDOS is required for execution of a FORTRAN program, provided no disk I/O is required and the user has taken the necessary steps to substitute his own routines for those normally used in EXbug.

In addition, MDOS FORTRAN easily interfaces with assembly language programs or routines which are assembled as relocatable modules. The output of the compiler are also relocatable modules.

1.2 APPLICATION

The FORTRAN language is especially useful in writing programs for applications that involve mathematical computations and other manipulation of numerical data. The name FORTRAN is derived from FORmula TRANslator.

With extensions incorporated into MDOS FORTRAN, many control type of applications become practical, including certain real-time applications. Three versions of MDOS FORTRAN are offered: a standard 6800 version, a standard 6809 version, and a 6800 version incorporating real-time features including a real-time operating

system. This manual applies to all versions except where indicated. Chapters 9, 10, and 11 apply only to the 6800 Real-Time version.

1.3 COMPILE PHASE OPERATION

Source programs, written in the FORTRAN language consist of a set of statements constructed by the programmer from the language elements described in this publication.

The compiler analyzes the source program statements and translates them into a machine language output called object programs which are relocatable modules. If the FORTRAN compiler detects errors in the source statements, it produces the appropriate diagnostic error message. The linking loader is utilized to create an absolute object program that can be executed by an MC6800 or MC6809 microprocessor (depending upon the version of MDOS FORTRAN being used).

1.3.1 MDOS COMMAND LINE

The MDOS FORTRAN compiler is invoked by the FORT command. This command and its parameters are defined as follows:

COMMAND NAME: FORT

PURPOSE: The FORT command processes source program statements written in MDOS FORTRAN language. The FORTRAN compiler, FORT, compiles these source statements into object programs. Under option control, a source listing is also produced.

FORMAT:

FORT [<delim><sfile>(.<suffix>][:<log drv>]][;<options>])

where <delim> is a valid command line delimeter

<sfile> may be one or more source program files (20 max.)

<suffix> is the file name suffix (.SA if not specified)

<log drv> is the logcal drive number of <sfile> (:0 if not specified)

<options> may be one or more of the compiler options shown in 1.3.2. Certain options are defaulted to being automatically specified or turned on. These options may be disabled or turned off by preceding the option letter with a minus sign (-)

1.3.2 COMMAND LINE OPTIONS

OPTION LETTER	ATTRIBUTE CONTROLLED BY OPTION	DEFAULT
A	Listing contains relative address	-A
B	Listing contains line sequence number	B
C	Source input is in card image format	-C
F	Fast subscript evaluation w/o error check	-F
H	Input intial heading from console	-H
I	All variable names are integer	-I
L	Print listing on line printer	-L
L=#LP,	Print listing on line printer (same as L)	-L
L=#CN,	Print listing on system console	-L
L=<fn>,	Print listing on disk file with name "fn" (Default suffix ".AL", default drive same as drive for first source file)	-L
M	Micromodule 14 or 14A in final system (Effective for 6800 Real-Time verson only)	-M
N=nnn,	Specifies maximum number of columns printed (50 <= nnn <= 120)	N=80
O	Produce object output in <sfile1.RO>	O
O=<fn>,	Produce object output in "fn" (Default suffix ".RO", default drive same as drive for first source file)	O
P=nn,	Specifies number of lines per page printed (10 <= nn <= 72) -P will inhibit paging	P=58
R	Compile for RAM/ROM dichotomy	-R
S	Symbol table listing	-S
X	Conditional compilation of "X" statements	-X
Y	Conditional compilation of "Y" statements	-Y

EXAMPLES:

=FORT CONVRT:1;LSA

will cause compilation of source file CONVRT.SA on drive 1, producing a compilation listing on the line printer with a symbol table, relative addresses and sequence line numbers, and an object file CONVRT.RO on drive 1.

=FORT PROG1.FS,PROG2.FS;L=#CN,S-0

will cause compilation of source files PROG1.FS and PROG2.FS on drive 0, with a compilation listing on the console with a symbol table and sequence line numbers, but no relative addresses will be displayed and no object output file will be produced.

1.3.3 OPTION H - HEADER LINE

This option allows entering of a header line up to 32 characters which will be displayed at the top of each printed page and also placed into the identification record of any relocatable object file produced.

1.4 SOURCE LINE COMPILER DIRECTIVES

In addition to the options which may be specified on the command line when calling up the compiler, certain options and printing directives are available if embedded in the source program. All are invoked by the use of a dollar sign ($) character in column 1 of the source file.

Directive	Meaning
$-L	Stop listing output.
$L	Start listing output again (will not override a "-L" option on the command line)
$P	Page to top of new page.
$n	Skip "n" lines on the listing, where n is 1 to 9.
$H	Change header to the 32 characters following the $H.

CHAPTER 2
ELEMENTS OF THE FORTRAN LANGUAGE

2.1 INTRODUCTION

The basic elements of the language are discussed in the following paragraphs. The actual FORTRAN statements in which these elements are used are discussed in subsequent chapters. The term program unit refers to a main program or a subprogram.

The order of a FORTRAN program unit is as follows:

1. Subprogram statement, if any.
2. EXTERNAL declarations, if any.
3. COMMON and DIMENSION statements, if any. They may be intermixed.
4. EQUIVALENCE statements, if any.
5. DATA statements, if any.
6. Executable statements.
7. END statement.

FORMAT and DATA statements may appear anywhere before the END statement. DATA statements, however, must follow any specification statements that contain the same variable or array names.

2.2 STATEMENTS

Source programs consist of a set of statements from which the compiler generates machine instructions, constants, and storage areas. A given FORTAN statement effectively performs one of three functions:

1. Causes certain operations to be performed (e.g., addition, multiplication, branching).

2. Specifies the nature of the data being handled.

3. Specifies the characteristics of the source program.

FORTRAN statements are composed of certain key words used with constants, variables, and expressions. The categories of FORTRAN statements are as follows:

1. Arithmetic Assignment Statements: These statements cause calculations to be performed or conditions to be tested. The result replaces the current value of a designated variable or subscripted variable.

2. Control Statements: These statements enable the user to govern the flow of and to terminate the execution of the object program.

3. Input/Output Statements: Those statements enable the user to transfer data between internal storage and the

terminal line printer, disk, or other device.

4. FORMAT Statement: This statement is used in conjunction with input/output statements to specify the form of a FORTRAN record.

5. DATA Initialization Statement: This statement is used to assign initial values to variables.

6. Specification Statements: These statements are used to declare the properties of variables and arrays.

7. Subprogram Statements: These statements enable the user to name and define functions and subroutines, which can be complied with the main program as one source file or as a separate file not existing with the main program.

2.3 CODING FORTRAN STATEMENTS

2.3.1 FREE FORMAT INPUT

The statements of a FORTRAN source program can be entered with an editor on a terminal. If a statement is too long for one line, it may be continued on successive lines by placing an "&" symbol in column 1 of each continuation line.

As many blanks as desired may be written between keywords and variable names to improve readability. Each keyword must have at least one blank following it. Blanks that are inserted in literal data are retained and treated as blanks within the data. Variable names, keywords, and numbers may not contain embedded blanks.

Comments for documentation purposes may be written in columns 2 through 72 of a line if the letter C or an asterisk (*) is placed in column 1. Comments may appear between FORTRAN statements; a comment line may not immediately precede a continuation line. Comments are ignored by the FORTRAN compiler except for listing. Comments may also be placed on a statement line if preceded by a semicolon (;).

The "C" or "*" indicating a comment record, the character "&" signifying statement continuation, the "$" for compiler directives, and an "X" or "Y" for conditional compliation must start in column 1. If an "X" is in column 1, the record is treated as a comment unless an "X" appears on the MDOS command line as an option when invoking the compiler. In this case, records with an "X" in column 1 will be complied. The same is true of a "Y" in column 1 and "Y" on the command line. Statement numbers ranging from 1-99999 also start in column 1 and are followed by at least one blank. All other statements may start anywhere from 2-72.

2.3.2 CARD IMAGE FORMAT INPUT

A "C" option specified on the command line when invoking the compiler allows the use of traditional "card image" type of input from a source file. With this, column 1 is the same in respect to the "C" or "*" for comments, "$" for compiler directives, and "X" or "Y" for conditional compilation. Columns 1-5 are otherwise used for statement labels (numbers) and column 6 for continuation. Statements must begin in column 7 or higher. Any non- blank character in column 6 will signify a continuation to the compiler.

2.4 CONSTANTS

A constant is a fixed, unvarying quantity. There are two classes of constants -- those that specify numbers (numerical constants), and those that specify literal data (literal constants).

Numerical constants may be integer or real numbers; and literal constants may be a string of alphanumeric and/or special characters.

2.4.1 Integer Constants

An Integer Constant is a whole number written without a decimal point. It occupies two bytes of memory. The allowable range is +32767 to -32768, and is interpreted as a base 10 (decimal) number. It must not contain embedded commas.

EXAMPLES

Valid Integer constants:

```
      0
     91
    173
 -21474
```

Invalid Integer Constants:

```
   27.      (contains a decimal point)
51459       (exceeds the allowable range)
5,396       (contains an embedded comma)
```

2.4.2 Hexadecimal Integer Constants

This version of FORTRAN permits the use of hexadecimal (base 16) constants wherever constants are permitted if the constant is prefixed with the dollar sign character ($). Also refer to FORMAT edit character "Z" for hexadecimal I/O.

Example:

```
KKA = $FCF4
CALL BO($8008,$3A)
```

2.4.3 Real Constants

A Real Constant has one of three forms: a basic real constant, a basic real followed by a decimal exponent, or an integer constant

followed by a decimal point. A real constant occupies four bytes of memory and is an approximation of a number. The precision using four bytes is approximately 6 decimal digits.

A basic real constant is a string of up to eight decimal digits with a decimal point.

The magnitude range of a real constant is 16E-64 through 16E63 (approximately 10E75), and including zero.

A real constant may be positive, zero, or negative (if unsigned, it is assumed to be positive) and must be within the allowable magnitude. It may not contain embedded commas. The decimal exponents permit the expression of a real constant as the product of a basic real constant or integer constant times 10 raised to a desired power.

EXAMPLES

Valid Real Constants:

```
-999.9999
   7.0E0
    7.E3
   7.0E3
   7E-03
```

Invalid Real Constants:

```
       1   (Missing decimal point or decimal exponent)
3,471.1    (Embedded comma)
    1.E    (Missing an integer constant after the E)
1.2E113    (E is followed by a 3-digit integer constant)
23.5E97    (Magnitude outside the allowable range)
```

2.4.4 Literal Constants

A Literal Constant is a string of alphanumeric and/or special characters, enclosed in apostrophes.

The string may contain any character except a single quote ('). Each character requires one byte of storage. The number of characters in the string, including blanks, may not be greater than 72.

Literals may be used in FORMAT, DATA, and assignment statements. Literals also may be used as the actual arguments in a CALL statement and are limited to two bytes.

EXAMPLES

```
'X-COORDINATE   Y-COORDINATE   Z-COORDINATE'
'3.14'
K = 'AB'
```

2.5 SYMBOLIC NAMES

Symbolic Names are from 1 through 6 alphanumeric characters (i.e., numerics 0 through 9 and uppercase alphabetic A through Z), the first of which must be alphabetic. No key word, such as GOTO, IF, FORMAT, etc... may be used as a symbolic name. All key words are considered reserved words.

Symbolic Names are used in a program unit (i.e., a main program or a subprogram) to identify elements in the following classes.

- An array and the elements of that array (see "ARRAYS")
- A variable (see "VARIABLES")
- An intrinsic function
- A FUNCTION subprogram (see "FUNCTION subprograms")
- A SUBROUTINE subprogram (see "SUBROUTINE subprograms")
- A MAIN program unit

Symbolic names must be unique within a class in a program unit and can identify elements of only one class with the exception of a function name.

A FUNCTION subprogram name must also be a variable name in the FUNCTION subprogram.

Once a symbolic name, or an external procedure name is used in any unit of an executable program, no other program unit of that executable program can use that name to identify an entity of these classes in any other way.

2.6 VARIABLES

A FORTRAN variable is a symbolic representation of a quantity that occupies a storage area. The value specified by the name is always the current value stored in the area.

For example, in the statement:

```
A = 5.0+B
```

both A and B are variables. The value of B is determined by some previous statement and may change from time to time. The value of A is calculated whenever this statement is executed and changes as the value of B changes.

2.6.1 Variable Name

Using meaningful variable names can serve as an aid in documenting a program. That is, someone other than the programmer may look at the program and understand its function. For example, the equation to compute the distance a car travels in a given period of time at a given rate of speed could be written:

```
X = Y * Z
```

Where "*" designates multiplication. However, it would be more meaningful to an individual reading this equation if the programmer had written:

```
DIST = RATE * TIME
```

EXAMPLES

Valid Variable Names:

```
B292S
RATE
VAR
```

Invalid Variable Names:

```
B292704     (Contains more than six characters)
4ARRAY      (First character is not alphabetic)
SI.X        (Contains a special character)
```

2.6.2 Variable types and Lengths

The type of a variable corresponds to the type of data the variable represents. Thus, an integer variable represents integer data and a real variable represents real data.

For each type of variable, there is a corresponding number of storage locations (bytes) that are reserved for the variable. The following list shows each variable type with its associated length:

Variable Type	Length (Bytes)
Integer	2
Real	4

2.6.3 Type Declaration

Type declaration by predefined specification is a convention used to specify variables as integer or real as follows:

1. If the first character of the variable name is I, J, K, L, M, or N, the variable is integer of length 2.

2. If the first character of the variable name is any other alphabetic character, the variable is real of length 4.

This convention is the traditional FORTRAN method of implicitly specifying the type of a variable as being either integer or real. In all examples that follow in this publication it is presumed that this specification applies.

The only exception to this convention in MDOS FORTRAN is that ALL names can be declared INTEGER by use of the "I" option at compile time.

2.7 ARRAYS

A FORTRAN array is a set of variables identified by a single variable name. A particular variable in the array may be referred top by its position in the array (e.g., first variable, third variable, seventh variable, etc.). Consider the array named NEXT which consists of five variables, each currently representing the following values: 273, 41, 8976, 59, and 2.

NEXT(1) is the location containing 273

NEXT(2) is the location containing 41

NEXT(3) is the location containing 8976

NEXT(4) is the location containing 59

NEXT(5) is the location containing 2

Each variable (element) in this array consists of the name of the array (i.e., NEXT) immediately followed by a number enclosed in parentheses, called a subscript quantity. the variables which the array comprises are called subscripted variables. Therefore, the subscripted variable NEXT(1) has the value 273; the subscripted variable NEXT(2) has the value 41, etc.

The subscripted variable NEXT(I) refers to the "Ith" subscripted variable in the array, where I is an integer variable that may assume a value of 1, 2, 3, 4, or 5.

To refer to any element in an array, the array name must be subscripted. In particular, the array name alone does not represent the first element.

Consider the following array named LIST described by two subscript quantities, the first ranging from 1 through 5, the second from 1 through 3:

	Column 1	Column 2	Column 3
ROW 1	82	4	7
ROW 2	12	13	14
ROW 3	91	1	31
ROW 4	24	16	10
ROW 5	2	8	2

uppose it is desired to refer to the number in row 2, column 3. his would be:

```
LIST(2,3)
```

hus, LIST(2,3) has the value 14 and LIST(4,1) has the value 24.

rdinary mathematical notation might use LIST to represent any lement of the array LIST. In FORTRAN, this is written as IST(I,J) where I equals 1, 2, 3, 4, or 5 and J equals 1, 2, or 3.

7.1 Declaring The Size And Type Of An Array

he size (number of elements) of an array is specified by the umber of subscript quantities of the array and the maximum value f each subscript quantity. This information must be given for ll arrays before using them in FORTRAN program so that an ppropriate amount of storage may be reserved. Declaration of his information is made by a DIMENSION statement or a COMMON tatement. These statements are discussed in detail in Chapter 7 SPECIFICATION STATEMENTS." The type of an array name is etermined by the conventions for specifying the type of a ariable name. Each element of an array is of the type specified or the array name.

7.2 Arrangement Of Arrays In Storage

rrays are stored in ascending storage locations, with the value f the first of the subscript quantities increasing most rapidly nd the value of the last increasing least rapidly.

or example, the array LIST, whose values are given in the revious example, is arranged in storage as follows:

```
82 12 91 24 2 4 13 1 16 8 7 14 31 10 2
```

he array named A, described by one subscript quantity which aries from 1 to 5, appears in storage as follows:

A(1) A(2) A(3) A(4) A(5)

The array named B, described by two subscript quantities with the first subscript quantity varying over the range from 1 to 5, and the second varying from 1 to 3, appears in ascending storage locations in the following order:

B(1,1) B (2,1) B(3,1) B(4,1) B(5,1)

B(1,2) B(2,2) B(3,2) B(4,2) B(5,2)

B(1,3) B(2,3) B(3,3) B(4,3) B(5,3)

Note that B(1,2) and B(1,3) follow in storage B(5,1) and B(5,2), respectively.

The following list is the order of a 3 dimensional array named C, described by three subscript quantities with the first varying from 1 to 3, the second varying from 1 to 2, and the third varying from 1 to 3:

C(1,1,1) C(2,1,1) C(3,1,1) C(1,2,1) C(2,2,1) C(3,2,1)

C(1,1,2) C(2,1,2) C(3,1,2) C(1,2,2) C(2,2,2) C(3,2,2)

C(1,1,3) C(2,1,3) C(3,1,3) C(1,2,3) C(2,2,3) C(3,2,3)

Note that C(1,1,2) and C(1,1,3) follow in storage C(3,2,1) and C(3,2,2), respectively.

2.8 SUBSCRIPTS

A subscript is an integer subscript quantity or a set of integer subscript quantities separated by commas, that is used to identify a particular element of an array. The number of subscript of the array with which the subscript is associated. A subscript is enclosed in parentheses and is written immediately after the array name. A maximum of three subscript quantities can appear in a subscript. Valid types are: integer constant, integer variable, or integer variable + integer constant.

The following restrictions apply to the construction of subscript quantities:

1. Subscript quantities may not contain arithmentic expressions that use any of the arithmetic operators: *, /, **.

2. Subscript quantities may not contain function references.

3. Subscript quantities may not contain subscripted names.

4. Variable subscripts must be integer only (not real).

5. The evaluated result of a subscript quantity should always

be greater than zero and less than or equal to the size of the corresponding dimension.

ubscript may have one of the following forms:

1. Positive integer constant e.e., 3, 21, 418
2. Integer variable
3. Integer variable plus/minus constant e., NOX+3, IX-5

EXAMPLES

Valid Subscripted Variables:

```
ARRAY(IHOLD)
NEXT(19)
MATRIX(I-5)
```

Invalid Subscripted Variables:

```
ARRAY(-9)    (Subscript may not be negative)
LOT(0)       (Subscript may never be zero)
ALL(X)       (Subscript may not be a real variable)
```

EXPRESSIONS

value of an arithmetic expression is always a number whose e is integer or real.

.1 Arithmetic Expressions

simplest arithmetic expression consists of a primary which may a single constant, variable, subscripted variable, function erence, or another expression enclosed in parentheses. The mary may be either integer or real.

an expression consisting of a single primary, the type of the mary is the type of the expression.

EXAMPLES

Primary	Type of Primary	Type of Expression
3	Integer constant	INTEGER*2
A	Real variable	REAL*4
3.14E3	Real constant	REAL*4
SIN(X)	Real function reference	REAL*4
(A*B+C)	Parenthesized real expression	REAL*4

More complicated arithmetic expressions containing two or more primaries may be formed by using arithmetic operators that express the computation(s) to be performed.

2.9.2 Arithmetic Operators

The arithmetic operators are as follows:

Arithmetic Operator	Meaning
**	Exponentiation
*	Multiplication
/	Division
+	Addition
-	Subtraction

2.9.3 Construction of Arithmetic Expressions

The following are the rules for construction of arithmetic expressions that contain arithmetic operators.

1. All desired computations must be specified explicitly. That is, if more than one primary appears in an arithmetic expression, they must be separated from one another by an arithmetic operator. For example, the two variables A and B will not be multiplied if written:

   AB

   If multiplication is desired, the expression must be written as follows:

A*B or B*A

2. No two arithmetic operators may appear in sequence in the same expression.

   For example, the following expressions are invalid:

   A*/B and A***B

   The expression A*-B is an exception and is treated as

   A*(-B)

   In effect, -B will be evaluated first and then A will be multiplied with it. (For further uses of parentheses, see rule 3.)

3. Order of Computation: Computation is performed from left to right according to the hierarchy of operations shown in the following list.

Operation	Hierarchy
Evaluation of functions	1st
Exponentiation (**)	2nd
Multiplication and division	3rd
Addition and subtraction	4th

   This hierarchy is used to determine which of two consecutive operations is performed first. If the first operator is higher than or equal to the second, the first operation is performed. If not, the second operator is compared to the third, etc. When the end of the expression is encountered, all of the remaining operations are performed in reverse order.

   For example, in the expression A*B+C*D**I, the operations are performed in the following order:

   a. A*B Call the result X (multiplication) (X+C*D**I)

   b. D**I Call the result Y (exponentiation) (X+C*Y)

   c. C*Y Call the result Z (multiplication) (X+Z)

   d. X+Z Final operation (addition)

   A unary minus has the highest hierarchy. Thus,

   A= -B is treated as A=0-B

   A= -B*C is treated as A=(-B)*C

A= -B+C is treated as A=(-B)+C

Parentheses may be used in arithmetic expressions, as in algebra, to specify the order in which the arithmetic operations are to be computed. Where parentheses are used, the expression within the parentheses is evaluated before the result is used. This is equivalent to the definition above since a parenthesized expression is a primary.

For example, the following expression:

B+((A+B)*C)+A**2

is effectively evaluated in the following order:

a.	(A+B)	Call the result X	B+(X*C)+A**2
b.	(X*C)	Call the result Y	B+Y+A**2
c.	B+Y	Call the result W	W+A**2
d.	A**2	Call the result Z	W+Z
e.	W+Z	Final operation	

4. The type and length of the result of an operation depends upon the type and length of the two operands (primaries) involved in the operation. Table 2-1 shows the type and length of the result of the operations +, -, *, and /.

TABLE 2-1. DETERMINING THE TYPE AND LENGTH OF THE RESULTS OF +, -, *, / OPERATIONS

+ - * /	INTEGER (2)	REAL (4)
INTEGER (2)	Integer (2)	Real (4)
REAL (4)	Real (4)	Real (4)

NOTE

When division is performed using two integers, the answer is truncated and an integer answer is given. For example, if I=9 and J=2, then the expression (I/J) would yield an integer answer of 4 after truncation.

Assume that the type of the following variables has been specified as follows:

Variable Names	Type	Length

C, D	Real Variable	4, 4
I, J, K	Integer Variable	2, 2, 2

Then the expression I*J/C**K+D is evaluated as follows:

Subexpression	Type and Length
I*J (Call the result M)	Integer of length 2
C**K (Call the result Y)	Real of length 4
M/Y (Call the result Z)	Real of length 4
Z+D	Real of length 4

Thus, the final type of the entire expression is real of length 4, but the type changed at different stages in the evaluation. Note that, depending on the values of the variables involved, the result of the expression I*J*C might be different from I*C*J.

5. The arithmetic operator denoting exponentiation (i.e., **) may only be used to combine the types of operands shown in Table 2-2.

TABLE 2-2. VALID COMBINATIONS WITH THE ARITHMETIC OPERATOR **

BASE		EXPONENT
Integer	**	Integer
Real	**	Integer

7.4 Logical Expressions

logical expression consists of two arithmetic expressions, which y be simple variables, connected by one of the following lational operators:

- .EQ. - equal
- .NE. - not equal
- .GT. - greater than
- .LT. - less than
- .GE. - greater than or equal to
- .LE. - less than or equal to

EXAMPLES

```
C.EQ.C
C+5.0.NE.21
```

```
(C+D)*E. GT. 50
```

It should be clearly understood here that arithmetic expressions involved in relational operations are evaluated first before the relational operation is applied.

Relational operations in turn may be connected by the use of the logical connectives .AND. and .OR.:

```
C. EQ. D. OR. E. EQ. F
C. NE. D. AND. E. GT. F. OR. G. EQ. H
```

Normally .AND. operations have a higher hierarchy than .OR. operations, thus C. EQ. D. AND. E. GT. F. OR. G. EQ. H is evaluated as

```
(C. EQ. D. AND. E. GT. F) .OR. G. EQ. H
```

However parentheses may be used to change the order or evaluation

```
C. EQ. D. AND. (E. GT. F. OR. G. EQ. H)
```

The meaning of a logical operation may be reversed by the modifier ".NOT.".

```
.NOT. (W. EQ. Y. AND. Z. EQ. V)
```

means everything but the intersection of W. EQ. Y. AND. Z. EQ. V

2.10 KEYWORDS

The following keywords are reserved by MDOS FORTRAN, and may not be used for any naming convention such as Symbolic names, Variable names, Array names, etc.

AND	ENDFILE	NOT
CALL	EQ	OPEN
CLOSE	EQUIVALENCE	OR
COMMON	EXTERNAL	PRINT
CONTINUE	FORMAT	PROGRAM
DATA	FUNCTION	READ
DECODE	GE	RETURN
DIMENSION	GO	REWIND
DO	GOTO	STOP
ELSE	GT	SUBROUTINE
ELSEIF	IF	TASK
ENCODE	LE	THEN
END	LT	TO
ENDIF	NE	WRITE

In addition, future releases of MDOS FORTRAN may implement some or all of the following list. Therefore, these names should be avoided in user programs if they are expected to be upward compatible:

ASSIGN	CONSTANT	OFF
BACKSPACE	DOUBLE	ON
BIT	IMPLICIT	PAUSE
BLOCK	INCLUDE	PRECISION
BYTE	INTEGER	REAL
CHARACTER	LOGICAL	SAVE

CHAPTER 3
ARITHMETIC ASSIGNMENT STATEMENT

3.1 GENERAL FORM

The general form is:

```
a = b
```

where: a is a subscripted or nonsubscripted variable
b is an arithmetic expression

This FORTRAN statement closely resembles a conventional algebraic equation; however, the equal sign specifies replacement rather than equality. That is, the expression to the right of the equal sign is evaluated, and the resulting value replaces the current value of the variable to the left of the equal sign.

Table 3-1 gives the conversion rules used for placing the evaluated result of arithmetic expression b into variable a.

3.2 ASSIGNMENT STATEMENTS

Assume that the type of the following variables has been specified as:

Variable Names	Type	Length
I, J, K	Integer Variables	2
A, B, C, D	Real Variables	4

Then the following examples illustrate valid arithmetic statments using constants, variables, and subscripted variables of different types:

Statements	Description
A = B	The value of A is replaced by the current value of B.
K = B	The value of B is truncated to an integer value and replaces the value of K.
A = I	The value of I is converted to a real value, and this result replaces the value of A.
J = J+1	The value of J is replaced by the value of J+1
A = C*D	The product of C and D replaces the value of A.

TABLE 3-1. Conversion Rules for Arithmetic Assignment Statement a = b

Type of a	Type of b	
	INTEGER	REAL
INTEGER	Assign	Fix and Assign
REAL	Float and Assign	Assign

1. Assign means transmit the resulting value, without change.

2. Fix means transform the resulting value to the form of a real constant and truncate the fractional portion.

3. Float means transform the resulting value to the form of a REAL number, retaining in the process as much precision of the value as a REAL number can contain.

CHAPTER 4
CONTROL STATEMENTS

4.1 INTRODUCTION

Normally, FORTRAN statements are executed sequentially. That is, after one statement has been executed, the statement immediately following it is executed. This section discusses the statements that may be used to alter and control the normal sequence of execution of statements in the program.

4.2 GO TO STATEMENTS

GO TO statements permit transfer of control to an executable statement specified by number in the GO TO statement. Control may be transferred either unconditionally or conditionally. The GO TO statements are:

1. Unconditional GO TO statement
2. Computed GO TO statement

4.2.1 Unconditional GO TO Statement

```
GO TO XXXX
   ----
```

Where: XXXX represents an executable statement number. The GO TO may be separated by a blank or written as GOTO.

This GO TO statement causes control to be transferred to the statement specified by the statement number. Every subsequent execution of this GO TO statement results in a transfer to that same statement. Any executable statement immediately following this statement must have a statement number; otherwise, it can never be referred to or executed.

EXAMPLE

```
      GO TO 25
10    A = B + C
        .
        .
        .
25    C = E**2
        .
        .
        .
```

In this example, each time the GO TO statement is executed,

ontrol is transferred to statement 25.

. 2. 2 Computed GO TO Statement

GENERAL FORM

GO TO (x1, x2, ..., xn)

where: i is a nonsubscripted integer variable
n has a range: 1<=n<=20

GO TO may be separated by a blank or written as GOTO

his stat[illegible]t causes control to be transferred to the statement umber[illegible] x2, x3,..., or xn, depending on whether the current of i is 1, 2, 3,..., or n, respectively. The index i is ecked at execution time to ensure it is within the range 1 <= i = n. If the i is outside that range, execution will continue at tatement following the computed GOTO. No error message will be ven.

EXAMPLE

```
      GOTO (25, 10, 7) ITEM

7     C = E**2+A

25    L = C

10    B + 21.3E02
```

this example, if the value of the integer variable ITEM is 1, tement 25 will be executed next. If ITEM is equal to 2, tement 10 is executed next, and so on.

ARITHMETIC CONTROL STATEMENTS

1 Arithmetic IF Statement

GENERAL FORM

```
IF (a) x1, x2, x3
```

Where: a is any arithmetic expression.
x1, x2, x3 are any executable statement numbers.

The arithmetic IF statement causes control to be transferred to the statement numbered x1, x2, or x3 when the value of the arithmetic expression (a) is less than, equal to, or greater than zero, respectively. The first executable statement following the arithmetic IF statement must have a statement number; otherwise, it can never be referred to or executed.

EXAMPLE

```
      IF (A(J,K)**3-B)10,4,30
        .
        .
        .
4     D = B + C
        .
        .
        .
30    C = D**2
        .
        .
        .
10    E = (F*B)/D+1
        .
        .
        .
```

In this example, if the value of the expression (A(J,K)**3-B) is negative, the statement numbered 10 is executed next. If the value of the expression is zero, the statement number 4 is executed next. If the value of the expression is positive, the statement numbered 30 is executed next.

4.3.2 Logical IF Statement

GENERAL FORM

```
IF (a) s
```

Where: a is any logical expression.
s is any valid executable FORTRAN statement except IF or DO.

The statement s is executed if the expression a is true; otherwise, the next executable statement following the logical IF statement is executed. The statement following the logical IF will be executed in any case after the statement s causes a transfer.

EXAMPLES

```
IF (FLAG1.OR.FLAG2) GO TO 123
IF (A*B.LT.1.23) CALL RATE
IF (.NOT.(A.LT.6.0.OR.B.GT.5.0) RETURN
```

If only a variable name is given as a, the variable will be considered true and the statement s will be executed if the named variable is positive (greater than or equal to zero). The variable will be considered false and the statement s will not be executed if the named variable is negative.

```
IF (MONDAY) GO TO 10
```

NOTE

If the expression (a) is real, a test for exact zero, or a test with the logical operator .EQ., may not be meaningful. If the expression involves any amount of computation, a very small value is more likely to result than a zero. For this reason, IF statements using real numbers should not be programmed to have a zero or .EQ. value.

4.3.3 Block IF Statement

An alternate extension to the Logical IF statement is the block IF statement. The block IF statement is used with the END IF statement and, optionally, with the ELSE or ELSE IF statements to form a structured programming sequence of execution.

GENERAL FORM

```
IF (a) THEN
```

Where: a is any logical expression.

The statement(s) following the THEN are executed if the expression s is true; otherwise the statement following the optional ELSE or ELSE IF is executed. If no ELSE or ELSE IF statement is present, then the statement following the END IF statement is executed next if the expression is false. The statement or statements following the THEN are executed until the ELSE or END IF is encountered, then control passes to the statement following the END IF.

lock IF statements may be nested. It is important, however, to

have an END IF statement paired with every IF - THEN combination.

The ELSE IF key word may contain the space, or may be written as ELSEIF. The remainder of the logical IF must continue on the same line as the ELSE IF (or on a following continuation line).

No other statements or key words may follow the THEN on a line.

The ELSE statement is used alone on a line, and there may not be any other key word following it (with the exception of the ELSE IF).

The END IF statement is used alone on a line and may be written ENDIF.

EXAMPLE:

```
IF (A.GT.B) THEN
    C=3.44
    D=C*A+6.21
  ELSE
    C=4.15
    D=C*B+7.07
  END IF
```

Note the use of indentation to aid in depicting the various levels of logic.

4.4 DO LOOPS

4.4.1 DO Statement

GENERAL FORM

	End of Range	DO Variable	Initial Value	Test Value	Increment
DO	x	i =	m1,	m2[,	m3]

Where: x is an executable statement number appearing after the DO statement.

i is a nonsubscripted integer value and cannot be a dummy.

m1, m2, and m3 are either unsigned integer constants greater than zero or unsigned nonsubscripted integer variable whose value is greater than zero. m2 may not exceed 32767 in value. m3 is optional; if it is omitted its value is assumed to be 1. In this case, the preceding comma must also be omitted. The DO and x must each be separated by a blank. Values m1, m2,

or m3, may not be an expression.

The DO statement is a command to execute, at least once, the statements that follow the DO statement, up to and including the statement numbered x. These statements are called the range of the DO. The first time the statements in the range of the DO are executed, i is initialized to the value m1 each succeeding time i is increased by the value m3. When, at the end of the iteration, i is equal to the highest value that does not exceed m2, control passes to the statement following the statement numbered x. Thus, the number of times the statements in the range of the DO are executed is given by the expression:

$$\left[\frac{m2 - m1}{m3}\right] + 1$$

The brackets represent the largest integral value not exceeding the value of the expression within the brackets. If m2 is less than m1, the statements in the range of the DO are executed once.

There are several ways in which looping (repetitively executing the same statements) may be accomplished when using the FORTRAN language. For example, assume that a manufacturer carries 1000 different machine parts in stock. Periodically, he may find it necessary to compute the amount of each different part presently available. This amount may be calculated by subtracting the number of each item used, OUT(I), from the previous stock on hand, STOCK(I).

EXAMPLE 1

```
      I=0
10    I=I+1
      STOCK(I)=STOCK(I)-OUT(I)
      IF(I-1000) 10,30,30
30    A=B+C
```

The first, second, and fourth statements required to control the obviously shown loop could be replaced by a single DO statement, as shown in Example 2.

EXAMPLE 2

```
      DO 25 I = 1, 1000
25    STOCK(I) = STOCK(I) - OUT(I)
      A = B+C
```

In Example 2, the DO variable, I is set to the initial value of 1. Before the second execution of statement 25, I is increased by the increment, 1, and statement 25 is again executed. After 1000 executions of the DO loop, I equals 1000. Since I is now equal to the highest value that does not exceed the test value, 1000, control passes out of the DO loop and the third statement is executed next.

EXAMPLE 3

```
      DO 25 I=1,10,2
      J=I+K
25    ARRAY(J)=BRAY(J)
      A=B+C
```

In Example 3, the DO variable, I, is set to the initial value of 1. Before the second execution of statement 25, I is increased by the increment, 2, and the second and third statements are executed a second time. After the fifth execution of the DO loop, I equals 9. Since I is now equal to the highest value that does not exceed the test value, 10, control passes out of the DO loop and the fourth statement is executed next.

4.4.2 Programming Considerations

1. The indexing parameters of a DO statement (i, m1, m2, m3) should not be changed by a statement within the range of the DO Loop.

2. There may be other DO statements within the range of DO statement. All statements in the range of an inner DO must be in the range of the outer DO. A set of DO statements satisfying this rule is called a nest of DO's.

EXAMPLE 1

```
      DO 50 I = 1,4-------------------------
      A(I) = B(I)**2                        ! Range of
      DO 50 J = 1, 5 ---! Range of          ! outer DO
50    C(J+1) = A(I)  ---! inner DO          !
                        -------------------
```

EXAMPLE 2

```
      DO 10 INDEX = L, M--------------------
      N = INDEX + K                         !
      DO 15 J = 1, 100, 2 -----             ! Range of
15    TABLE(J) = SUM(J,N)-1     ! Range of  ! outer DO
                          ----  outer DO    !
10    B(N) = A(N)---------------------------
```

3. At transfer out of the range of any DO loop is permissible at any time.

4. Never transfer into the middle of a DO loop with a GO TO.

5. The extended range of a DO is defined as those statements in the program unit containing the DO statement that are executed between the transfer out of the innermost DO of a nest of DO's and the transfer back into the range of this innermost DO. The following restrictions apply:

   - Transfer into the range of a DO is permitted only if such a transfer is from the extended range of the DO.

   - The extended range of a DO statement must not contain another DO statement that has an extended range if the second DO is within the same program unit as the first.

   - The indexing parameters (i, m1, m2, m3) cannot be changed in the extended range of the DO.

   Note that a statement that is the end of the range of more than one DO statement is within the innermost DO. The statement label of such a terminal statement may not be used in any GO TO or arithmetic IF statement that occurs anywhere but in the range of the most deeply contained DO with that terminal statement.

6. The indexing parameters (i, m1, m2, m3) may be

changed by the statements outside the range of the DO statement only if no transfer is made back into the range of the DO statement that uses those parameters.

7. The last statement in the range of a DO loop (statement x) must be an executable statement. It cannot be a GO TO statement of any form, or a STOP, RETURN, arithmetic IF statement, or another DO statement.

8. The use of, and return from, a subprogram from within any DO loop in a nest of DO's is permitted.

4.5 CONTINUE Statement

GENERAL FORM

```
CONTINUE
```

CONTINUE is a dummy statement that may be placed anywhere in the source program without affecting the sequence of execution. It may be used as the last statement in the range of a DO in order to avoid ending the DO loop with a GO TO, STOP, RETURN, arithmetic IF, or another DO statement.

EXAMPLE :

```
      .
      .
      .
      DO 30 I=1,20
7     IF (A(I)-B(I)) 5,30,30
5     A(I)=A(I)+1.0
      B(I)=B(I)-2.0
      .
      .
      .
      GO TO 7
30    CONTINUE
      C=A(3)+B(7)
      .
      .
      .
```

In Example 1, the CONTINUE statement is used as the last statement in the range of the DO, to avoid ending the DO loop with the statement GO TO 7.

EXAMPLE 2

```
      DO 30 I=1,20
      IF (A(I)-B)I)) 5,40,40
5     A(I)=C(I)
      GOTO 30
40    A(I)=0.0
30    CONTINUE
      C=A(3)+B(7)
```

:n Example 2, the CONTINUE statement provides a branch point :nabling the programmer to bypass the execution of statement 40.

:.6 STOP Statement

GENERAL FORM

```
      STOP
```

he STOP statement defines the logical end of an executing rogram. Its execution causes the FORTRAN program to print the ord "STOP" on the console terminal and return to the operating ystem. This statement may be used any number of times in a rogram or sub-program or may be omitted.

.7 END Statement

GENERAL FORM

```
      END
```

ıe END statement is a nonexecutable statement that defines the ıd of a source program or source subprogram for the compiler. ıysically, it must be the last statement of each program or ıbprogram. The END statement replaces a STOP statement at the ıysical end in a program or replaces a RETURN statement at the ıysical end of a sub-program.

CHAPTER 5
INPUT/OUTPUT STATEMENTS

Input/output statements are used to transfer and control the flo of data between internal storage and an input/output device, suc as a terminal or disk storage unit.

5.1 INPUT/OUPUT LIST

Input/output statements in FORTRAN are primarily concerned wit the transfer of data between storge locations defined in a FORTRA program and records external to the program. On input, data i taken from a record and placed into storage locations that are no necessarily contiguous. On output, data is gathered from divers storage locations and placed into a record. An I/O list is use to specify which storage locations are used. The I/O list ca contain variable names, unsubscripted array names, subscripted arra names, or array names accompanied by indexing specifications in form called an implied DO. No function references or arithmeti expressions are permitted in an I/O list.

If an unsubscripted array name appears in the list, the entir array is transmitted in the order in which it is stored. (If th array has more than one dimension, it is processed by ascendin storage locations. An example is given in Paragraph 2.7.2, "Arrangement of Arrays in Storage".)

If an implied DO appears in the I/O list, the elements of th array(s) specified by the implied DO are transmitted. The implie DO specification is enclosed in parentheses. Within th parentheses there are one or more subscripted array names, separated by commas with a comma following the last name, follow by indexing paremeters i=m1, m2, m3. The indexing parameters ar as defined for the DO statement. Their range is the list of th DO-implied list and, for input lists, i, m1, m2, and m3 may appea within that range only in subscripts.

Example: A is a variable; B, C, and D are 1-dimensio arrays, each containing 20 elements. Th statement:

```
PRINT 998,A,B,(C(I),I=1,4),D(4)
```

writes the current value of variable A, the entir array B, the first four elements of the array C, and the fourth element of D.

Implied DO's can be nested, if required. For example, th following would be written to read an element into array B afte values are read into each row of a 10x20 array A:

```
READ 998,((A(I,J),J=1,10),B(I),I=1,20)
```

The order of the names in the list specifies the order in whic

the data is transferred between the record and the storage locations.

Data is transmitted under control of a FORMAT statement controlling the transmission of the data in the record from a form that can be read by the programmer to a coded form that satisfies the needs of machine representation. The transformation for input takes the character codes and constructs a machine representation of an item. The output transformation takes the machine representation of an item and constructs character codes suitable for output. Most transformations involve numeric representations that require base conversion. For formatted I/O the programmer must include a FORMAT statement in the program, and must give the statement number of the FORMAT statement in each READ or WRITE statement.

5.2 SEQUENTIAL INPUT/OUTPUT STATEMENTS

There are four sequential input/output statements: READ, WRITE, PRINT, and REWIND. The READ and WRITE statements initiate the transfer of records of sequential files or console terminal data transfer. The PRINT statement is used to transfer data to the console terminal. The REWIND statement controls the positioning of the file. In addition to these four statements, the FORMAT statement, although not an input/output statement, is used with the READ, WRITE, and PRINT statements.

Before data can be read from or written to a disk file, the file must be opened. When file I/O is complete, the file must be closed before the program is terminated. See Paragraph 5.8, OPEN/CLOSE Statements for a discussion of these.

The following reference chart indicates the MDOS FORTRAN pre-assigned file reference number:

NUMBER	ASSIGNMENT or USAGE
99	Dummy device. Buffer I/O
100	Console keyboard
101	Console printer or display
102	Line printer
103	Reserved

5.3 READ STATEMENT

GENERAL FORM

```
READ a, list
READ (b,a) list
```

Where: a is the statement number of the FORMAT statement describing the record(s) being

read.

b is an unsigned integer constant o integer variable that is in the ra to 255 and represents a file refe number.

list is an I/O list of the variables

The READ statement may take two forms. The value of a must al be specified, but b can be omitted. The form READ a, list is to read data from the console according to the specification FORMAT statement a.

The form READ (b,a) list is used to read data from file number into the variables whose names are given in the list. The data transmitted from the file to memory according to specifications in the FORMAT statement, which is statement num a.

EXAMPLE 1

```
READ(5,98)A,B,(C(J,K),J=1,10)
```

The above statement causes input data to be read from the de file number 5 into the variables A, B, C(1,K), C(2,K),...,C(10,K in the format specified by the FORMAT statement whose statemer number is 98.

EXAMPLE 2

```
READ 98,A,B,(C(J,K),J=1,10)
```

The above statement causes input data to be read from the console terminal keyboard into the variables A, B, C(1,K), C(2,K,...,C(10,K) in the format specified by the FORMAT statement whose statement number is 98.

EXAMPLE 2

```
READ (100,98)A,B,(C(J,K),J=1,10)
```

The above statement reads data from the console terminal as in the preceding example.

Refer to Paragrah 5.9.1 for further disk file information.

REREAD CAPABILITY: Sometimes it is desired to have records in a data file which are not uniform in format. This feature allows a re-read of the I/O record buffer without reading in a new record. Use file number 99 to accomplish this.

EXAMPLE: READ(7,900)A,B,J

```
READ (99,901)C,K,L
```

Allows reading from file number 7 under format number 900 and rereading the same record under format number 901.

5.4 WRITE STATEMENT

GENERAL FORM

```
WRITE (b,a)list
```

Where:

- a is the statement number of the FORMAT statement describing the record(s) being written.
- b is an unsigned integer constant or an integer variable that is in the range 1 to 255 and represents a file reference number.
- list is optional and is an I/O list of variables that will be written to disk according to the FORMAT a.

The statement WRITE (b,a) list is used to write data into the file whose reference number is b from the variables whose names are given in the list. The data is transmitted from memory to the file according to the specifications in the FORMAT statement, whose statement number is a.

EXAMPLE

```
WRITE (10,75)A,(B(J,3),J=1,10,2),C
```

The above statement causes data to be written from the variables A, B(1,3), B(3,3), B(5,3), B(7,3), B(9,3), and C to file number 10 in the format specified by the FORMAT statement whose statement number is 75. If the file number were 101 instead of 10, the data would have been printed at the console; or if it were 102, data would have been printed on the line printer.

5.5 PRINT STATEMENT

GENERAL FORM

```
PRINT a,list
```

Where:

- a is the statement number of the FORMAT statement describing the record(s) being printed.
- list is optional and is an I/O list of variables that will be printed according to the FORMAT a.

The statement "PRINT a,list" is used to print data at the consol from the variables whose names are given in the list. The data i transmitted from memory to the console according to th specifications in the FORMAT statement, whose statement number i a.

EXAMPLE

```
PRINT 75,A,(B(J,3),J=1,10,2),C
```

The above statement causes data to be written from the variables A, B(1,3), B(3,3), B(7,3), B(9,3), and C to the console in the format specified by the FORMAT statement whose statement number is 75.

5.6 ENCODE/DECODE STATEMENTS

These statements are used to re-format data which is being stored in variables. ENCODE allows writing to a buffer under format control a list of variables, the same as a WRITE statement except that the characters remain in the buffer and not sent to an output device. DECODE then allows reading of that buffer under a different format control. It is much the same as a READ statement except that the characters are already in a buffer and therefore no access of an input device is required.

GENERAL FORM

```
ENCODE fsn,list

DECODE fsn,list

Where:    fsn  is the FORMAT Statement Number

          list is the variable list
```

MDOS FORTRAN uses the I/O buffer which has a maximum length of 1 characters for ENCODE/DECODE operations. Therefore, the form statement must not contain any slash characters or exceed t maximum buffer length. Since this buffer is shared with other I/ the DECODE statement should immediately follow the ENCO statement in the program.

EXAMPLE:

```
      I='AB'
      J='CD'
      ENCODE 1,I,J
      DECODE 2,A
1     FORMAT(2A2)
2     FORMAT(A4)
```

n the above example, the variable "A" will contain the literal ABCD" after execution of the statements. In the following xample, a numeric integer is changed to a literal (that is, ariable K contains the numeric 16-bit representation of the umber 73, while L will contain the ASCII characters $37 and $33 fter execution).

EXAMPLE:

```
        K=73
        ENCODE 3,K
        DECODE 4,L
3       FORMAT(I2)
4       FORMAT(A2)
```

5.7 EOFTST (END-OF-FILE TEST) SUBROUTINE

This subroutine is used to test for END-OF-FILE conditions on files. Normally, a read encountering an END-OF-FILE terminates the run in an error condition.

GENERAL FORM

```
CALL EOFTST(IUNIT,IFLAG)
```

Where: IUNIT is an unsigned integer constant or an integer variable n the range 1<=IUNIT<=255, and represents a file reference number (FORTRAN UNIT NUMBER) to be tested for an end-of-file condition. The numbers 99 through 103 are reserved for special use.

IFLAG is an integer variable which is set to two (2) if an END-OF-FILE has been encountered; otherwise, it is set to one (1).

EXAMPLE

```
    DIMENSION IN(7),IOUT(7)
    DATA IN/'DI','SK','DA','TA','.S','A:','1'/
    DATA IOUT/'DI','SK',':1'/
    OPEN (10,IN,1)
    OPEN (11,IOUT,2)
    CALL EOFTST(10,MN)
40  READ (10,100) I,J,K,L
```

```
100   FORMAT(4I3)
      CALL EOFTST(10,MN)
      GO TO (50,60),MN
50    PRINT 100, I,J,K,L
      WRITE(11,100) I,J,K,L
      GO TO 40
60    CALL DELF(10)
      CLOSE (11)
      END
```

When using EOFTST, an END-OF-FILE status is maintained for e device. Thus, the test can be performed on as many differ devices as desired during one program.

The first call EOFTST, which must occur after the file is open sets the END-OF-FILE indicator for this device to prevent the from terminating when an END-OF-FILE condition is encountered o READ. If this condition is encountered before the first execut of a call EOFTST, the run terminates. If the first call EOFTST the above example is omitted and the file is empty, the terminates.

Upon return from the second call EOFTST, MN is set to one (1) an END-OF-FILE has not been encountered, or two (2) if END-OF-FILE induciation has been encountered.

5.7 REWIND STATEMENT

GENERAL FORM

REWIND b

Where: b is an unsigned integer constant integer variable that is in the ran 1<b<255 and represents a fi reference/number.

The REWIND statement causes a subsequent READ or WRITE stateme referring to b to read data from or write data into the fir record of file number b.

5.9 FORMAT STATEMENT

GENERAL FORM

xxxxx FORMAT (c1, c2, ... , cn)

Where: xxxxxis a statement number (1 through digits.)

c1, c2, ... , cn are format codes.

The format codes are:

aIw	Describes integer data fields.
aZw	Describes integer hexadecimal base data fields.
aEw.d	Describes real data fields.
aFw.d	Describes real data fields.
aAw	Describes alphameric data fields.
aRw	Describes alphameric data fields.
BN	Indicates a blank is ignored in numeric input field.
BZ	Indicates a blank is a zero in numeric input field.
Bm	Describes a bit data field.
'Literal'	Transmits literal data.
wX	Indicates that a field is to be filled with blanks on output or skipped on input.
a(...)	Indicates a group format specification.

Where: a is optional and is an unsigned integer constant used to denote the number of times the format code is to be used. If a is omitted, the code is used only once.

w is an unsigned nonzero integer constant that specifies the number of characters in the field.

d is an unsigned integer constant specifying the number of decimal places to the right of the decimal point; i.e., the fractional portion.

(...) is a group format specification. Within the parentheses are format codes separated by commas or slashes. Group format specifications can be nested to a level of two. The a preceding this form is called a group repeat count. Note: Both commas and slashes can be used as separators between format codes (see Paragraph 5.9.1, "Various Forms of a FORMAT Statement").

m is a bit mask.

The FORMAT statement is used in conjunction with the I/O list the READ, PRINT, and WRITE statements to specify the structure FORTRAN records and the form of the data fields within records. In the FORMAT statement, the data fields are descri with edit codes, and the order in which these edit codes specified gives the structure of the FORTRAN records. The list gives the names of the data items to make up the record. length of the list in conjunction with the FORMAT statem specifies the length of the record (see Paragraph 5.7. Throughout this paragraph, the examples show console input output. However, the concepts apply to all input/output media.

The following list gives general rules for using FOR statements:

1. FORMAT statements are not executed; their function is supply information to the object program. They may placed anywhere in the source program af specification statements.

2. When defining a FORTRAN record by a FORMAT statement, is important to consider the maximum size record allou on the input/output medium. For example, if a FORTR record is to be printed, the record should not be long than 80 characters.

3. If the I/O list is omitted from the READ, WRITE, PRINT statement, a record is skipped on input, or blank record is inserted on output.

4. Types I, Z, and B are valid only with integer variable Types E and F are valid only with real variables.

5.9.1 Various Forms of a FORMAT Statement

All of the edit codes in a FORMAT statement are enclosed in pair of parentheses. Within these parentheses, the edit codes a delimited by the separators: comma and slash.

Execution of a READ, WRITE, or PRINT statement initiates form control. Each action of format control depends on informati provided jointly by the I/O list -- if one exists -- and the ed specification. There is no I/O list item corresponding to t edit descriptors X and literals enclosed in apostrophes. The output information directly to the record.

Whenever an I, E, F, Z, B, R or A code is encountered, form control determines whether or not there is a corresponding elemer in the I/O list. If there is such an element, appropriatel converted information is transmitted. Format control terminate when these codes are encountered and there is no correspondir data item in the I/O list.

If, however, format control reaches the last outer righ

parenthesis of the edit specification and another element is specified in the I/O list, control is transferred to the group repeat count of the group edit specification terminated by the last right parenthesis that precedes the right parenthesis ending the FORMAT statement.

The question of whether or not there are further elements in the I/O list is asked only when an I, E, F, Z, B, R, or A, or the final right parenthesis of the edit specification, is encountered. Before this is done, X, literals enclosed in apostrophes, and slashes are processed.

If there are fewer elements in the I/O list than there are edit codes, the remaining edit codes are ignored.

5.9.1.1 COMMA

The simplest form of a FORMAT statement is the one shown at the beginning of Paragraph 5.7 with the edit codes, separated by commas, enclosed in a pair of parentheses. One FORTRAN record is defined by the beginning of the FORMAT statement (left parenthesis) to the end of the FORMAT statement (right parenthesis).

5.9.1.2 SLASH

A slash is used to indicate the end of a FORTRAN record format. For example, the statement:

```
25    FORMAT (I2,F6.2/E10.3,F6.2)
```

describes two FORTRAN record formats. The 1st, 3rd, etc. records are transmitted according to the format I2, F6.2, and the 2nd, 4th, etc. records are transmitted according to the format E10.3, F6.2.

Consecutive slashes can be used to introduce blank output lines. If there are "n" consecutive slashes at the beginning or end of a FORMAT statement, "n" blank lines are inserted between output records. If "n" consecutive slashes appear anywhere else in a FORMAT statement, the number of blank lines inserted is "n-1". For example, the statement:

```
30    FORMAT (1X,10I5//1X,8E14.5)
```

describes three FORTRAN record formats. On output, it causes double spacing between the line written with format 1X,10I5 and the line written with the format 1X,8E14.5.

5.9.2 I Edit Code

The I edit code is used in transmitting integer data. For example, if a PRINT statement refers to a FORMAT statement containing I edit codes, the input data is assumed to be stored in

internal storage in integer format.

INPUT — Leading, embedded, and trailing blanks in field of the input record are ignored unles: BZ has been specified previously in the FORI statement, in which case all blanks treated as zeros.

OUTPUT — If the number of significant digits and si required to represent the quantity in t storage location is less than w, the leftmc print positions are filled with blanks. If is greater than w, the number is printed a expanded to the right (w is overridden).

5.9.3 Z Edit Code

The Z edit code is the same as the I edit code, except th numeric data is interpreted as hexadecimal instead of decimal. fields wider than necessary to print the number, leading zer will be output. For example, with a Z4 specification, t hexadecimal number 3C4 will be printed as 03C4.

5.9.4 E and F Edit Codes

The E and F edit codes are used in transmitting real data. T data must not exceed the maximum magnitude for a real constant.

INPUT — Input must be a real number which, optionall may have an exponent. The decimal point m be omitted. If it is present, its positi overrides the position indicated by the portion of the format field descriptor, a the number of positions specified by w mu include a place for it. Each data item mu be right justified in its field. Leading trailing, and embedded blanks are ignorec These two format codes are interchangeable fc input. It makes no difference, for example whether E or F is used to describe a fiel containing 12.42E00.

OUTPUT — For data written under an E format code output consists of an optional sign (require for negative values), a decimal point, th number of significant digits specified by d and an E exponent requiring four positions The w specification should provide for al these positions, including the one for a sig when the output value is negative. I additional space is available, a leading zer may be written before the decimal point.

For data written under an F format code,

should provide sufficient spaces for an integer segment, if it is other than zero, a fractional segment containing d digits, a decimal point, and a sign. If too few spaces are available, w will be overridden and the full number printed. If excess positions are provided, the number is preceded by blanks.

For E and F edit codes, fractional digits in excess of the number specified by d are dropped.

Edit codes E, F, and I: If the columns required on a WRITE exceeds the specified number of columns in the format statement, FORTRAN will allow writing of the full number, altering the format to fit the number. Thus, with a format of F5.2, the value 1234.567 would be printed as 1234.56 (normally requiring a format of F7.2). Digits are not truncated. A column is required for the sign in the E, F, and I formats if it is a minus.

Left justifying numeric values on printout: By 'underformatting', it is possible to left justify numeric values due to the expanding formt width feature mentioned above. This could be quite useful in output such as:

```
There are 3 items in inventory.          (use of I1 format)
There are 59712 items in inventory.      (use of I1 format)
```

5.9.5 Examples of Numeric Format Codes

The following examples illustrate the use of the format codes I, F, and E.

EXAMPLE 1

```
75 FORMAT (1x,I3,F5.2,E10.3,E10.3)
PRINT 75, N,A,B,C
```

1. Four fields are described in the FORMAT statement, and four variables are in the I/O list. Therefore, each time the PRINT statement is executed, one line is printed on the console terminal.

2. When a line is printed, the number in integer format in location N is printed in the first field of the line (three columns). The number in the second field of the line (five columns) is printed in real format, and comes from location A, etc.

3. If there were one more variable in the I/O list, say M, another line would be printed, and the information in the first three columns of that line would be printed in integer format and obtained from location M. The rest of the line would be blank.

4. If there were one fewer variables in the list (say C i omitted), no number would be printed according to th format E10.3.

EXAMPLE 2

Assume that the following statements are given:

```
    76   FORMAT (1X,F6.2,E12.3,I5)
         PRINT 76,A,B,N
```

and that the variables A, B, and N have the following values:

A	B	N
34.40	123.380E+02	31
31.10	11546.10E+02	130
0.00	834.621E-03	428
1.139	83.121E+06	0

Then the following lines are printed:

```
 34.40   0.123E+05   31
 31.10   0.115E+07  130
  0.00   0.834E+00  428
  1.13   0.831E+08    0
```

5.9.6 A and R Format Codes

The A and R format codes are used in transmitting data that is stored internally in character format. The number of characters transmitted under A or R format code is limited to two characters per integer variable or four characters per real variable. Each character is stored in ASCII. Numeric data is converted digit by digit into ASCII, rather than the entire numeric field being converted into a single binary number. Thus, the A and R format codes can be used for numeric fields, but not for numeric fields requiring arithmetic.

The difference between the A and R format codes is that the A code left justifies characters in storage, while the R code right justifies the characters in storage. For example, if a single ASCII character were stored in an integer variable (2 bytes), the character would actually be stored in the most sighificant (lower address) byte under the A format code.

EXAMPLE 1

```
9900 FORMAT (A2,A1)
     READ 9900,I,K
```

The following is entered after the ? when the program is executed:

```
? ABC
```

The AB will be stored in I, and C will be left justified and stored in K.

If it is printed with a different FORMAT:

```
9910 FORMAT (2A1)
     PRINT 9910,I,K
```

the following will be printed at the console:

```
AC
```

EXAMPLE 2

```
        DIMENSION I(5)
        I(1) = 'TH'
        I(2) = 'E'
        I(3) = 'QU'
        I(4) = 'IC'
        I(5) = 'K'
        PRINT 9900,I
  9900  FORMAT (5A2)
```

"THE QUICK" will be printed at the console.

5.9.7 X Format Code

The X format code specifies a field of w characters to be skipped on input or filled with blanks on output.

EXAMPLE:

```
5 FORMAT (110,10X,4110)
     READ (5,5) I,J,K,L,M
```

The first ten chracters of the input record are read into variable I, the next ten characters are skipped over, and the next four fields of ten characters each are read into the variables J, K, L, and M.

5.9.8 B Format Code

The B Format code allows writing of one or more bits of a byte. It is intended for use with an I/O device driver routine such as described in Chapter 10.

On output, three bytes appear in the I/O buffer as a result of each B code encountered in the FORMAT statement. The first byte is a flag byte of $00. The second byte is the bit mask as specified by the "m" of Bm Format specification. The third byte

is the least significant byte of data from the integer vari
specified in the I/O list.

The intention is to have the device driver recognize the 00
byte as it scans the buffer, and then utilize the mask and
bytes to alter only the bits specified as "1's" in the mask by

As an example, let's say the variable J has the hexedecimal v
of 0A265. If the format specifier for this variable were
then the following would be output on a write statement to the
buffer:

00 C3 65

The mask of $C3 (or 11000011 in binary) specifies that only
0, 1, 6, and 7 be altered by the data of $65 (or 01100010
binary). The result would be that bit 0 would be a 1, bit 1 w
be a 0, bit 6 would be a 1, and bit 7 would be a 0 on the ou
device.

This format code will find use in interfacing with a device
as a PIA (MC6821) or Micromodule boards containing 32 I/O li
Note that the device driver must interpret the data in the
buffer resulting from the use of this format code.

5.9.9 LITERAL DATA

Literal data can appear in a FORMAT statement as a string encl
in apostrophes.

```
25 FORMAT (' 1975 INVENTORY REPORT')
```

No item in the I/O list corresponds to the literal data. The
is written directly from the FORMAT statement. (The FOI
statement can contain other types of format codes
corresponding variables in the I/O list). Example:

```
8 FORMAT ('MEAN AVERAGE: ',F9.4)
PRINT 8,AVRGE
```

The following record is written if the value of AVRGE is 12.34!

```
MEAN AVERAGE:  12.3456
```

5.9.10 Group Format Specification

The group format specification is used to repeat a set of fo
codes and to control the order in which the format codes are u

The group repeat count a is the same as the repeat indicator
which can be palced in front of other format codes. The follo
statements are equivalent:

```
10 FORMAT      (I3,2(I4,I5,I6)
```

```
10 FORMAT       (I3, I4, I5, I4, I5, I6)
```

Group repeat specifications control the order in which format codes are used since control returns to the last group repeat specification when there are more items in the I/O list than there are format codes in the FORMAT statement (see Paragraph 5.7.1, "Various Forms of a FORMAT Statement"). Thus, in the previous example, if there were more than six items in the I/O list, control would return to the group repeat count 2 which precedies the specification (I4, I5).

The format codes within the group repeat specification can be separated by commas and slashes. The following statement is valid:

```
40 FORMAT       (2I3/(3F6.2, F6.3/E10.3, 3E10.2)
```

The first record is transmitted according to the specification 2I3; the second, fourth, etc., records are transmitted according to the specification 3F6.2, F6.3, and the third, fifth, etc., records are transmitted according to the specification E10.3, 3E10.2, until the I/O list is exhausted.

5.9.11 Free Format Input

Data may be read from the console in free field, comma-separated input by specifying an empty format. For example,

```
998 FORMAT ( )
```

Data read in this manner will be converted to integer or real, depending upon the mode of the receiving variable. The values to be typed for the variables must be in the proper format for a real or integer constant, and are separated by commas.

EXAMPLE

```
      READ 998, I, J, X

998 FORMAT ( )
```

The values may be input in the following form:

```
 3, 5, 8, 3
-3, 6, 5, 8
etc.
```

Free Format Input may NOT be used for alphanumeric data.

5.10 OPEN/CLOSE STATEMENTS

The OPEN and CLOSE statements give the FORTRAN programmer control of disk file handling. With the MDOS operating system, one or

more disk files can be open at a given time.

5.10.1 OPEN/CLOSE Statement Arguments

GENERAL FORM

```
OPEN (IUNIT, IFILE, IMODE)
CLOSE (IUNIT)
```

Where: IUNIT is an unsigned integer constant or an integer variable in the range a IUNIT 255, and represents a file reference number (FORTRAN unit number). (99 through 103 are reserved for specil use.)

IFILE is a 1-7 element integer array containing the file name (in standard MDOS format) as a 1-13 literal ASCII character string or a single integer variable containing the file name as a 1-2 literal ASCII character string. The file name in standard MDOS format is as follows:

FILENAME.SX:N

Where "FILENAME" is a 1-8 character name, the period (".") is the suffix delimiter, "SX" is a 1-2 character suffix, the colon (":") is the logical drive delimiter, and "N" is the logical drive number.

IMODE is an unsigned integer constant or an integer variable specifying the mode with which the file is to be opened.

1 = input mode
2 = output mode
3 = append mode

No defaults are assumed for any of the arguments; therefore, all arguments must be specified. Note that three (3) arguments are required for OPEN, while only one (1) is required for CLOSE. While no defaults are assumed for any arguments, the suffix and/or logical drive number portion(s) of IFILE will default to "SA" and "0", respectively, if omitted.

Additional information about the argments is in Paragraph 8.7, "Arguments in a Function or Subroutine Subprogram".

5.10.2 OPEN/CLOSE Programming Considerations

The statement OPEN (IUNIT, IFILE, IMODE) is used to open a file for input (read) or output (write). To open a file for input, the

file name must already exist in the directory. If the file is no found in the directory, an appropriate MDOS error is returned. T open a file for output, the file name must not be in th directory. If the file name already exists, or if there is n more room in the disk directory or the disk file area, a appropriate MDOS error is returned.

The statement CLOSE (IUNIT) is used to close a disk file afte input from or output to a file is complete. If the file wa opened for input, a flag will be set to indicate the file is nc longer open. If opened for output, an end-of-file record is written, the directory is updated, and a flag is set to indicate the file is no longer open. All files should be closed before exiting from the FORTRAN program.

5.10.3 OPEN/CLOSE Examples

The following examples illustrate several OPEN/CLOSE CALLS. The examples assume that I and K have been assigned valid values in previous assignment or data statements.

EXAMPLE 1:

```
OPEN (I,'FN',K)
CLOSE (I)
```

EXAMPLE 2:

```
J='FN'
OPEN (I,J,K)
CLOSE (I)
```

EXAMPLE: 3

```
DIMENSION J(7)
DATA J/'FL','NA','ME'/


OPEN (I,J,K)
CLOSE (I)
```

EXAMPLE 4:

```
DIMENSION J(7)
J(1)='FI'
J(2)='LE'
J(3)='NA'
J(4)='ME'


OPEN (I,J,K)
CLOSE (I)
```

In the four examples above, the OPEN call will result in the default suffix (SA) and the default logical drive number (0) being used, since the suffix and logical drive are not explicitly provided.

EXAMPLE 5:

```
DIMENSION J(7)
DATA J/'FILE.FT'/

OPEN (I,J,K)
CLOSE (I)
```

EXAMPLE 6:

```
DIMENSION J(7)
J(1)='FI'
J(2)='LN'
J(3)='AM'
J(4)='E. '
J(5)='FT'
      .
      .
      .
OPEN (I,J,K)
CLOSE (I)
```

In the two examples above, the OPEN call will result in the default logical drive number being used. The suffix for Examples 5 and 6 is FT.

EXAMPLE 7:

```
DIMENSION K(7)
J(1)='FI'
J(2)='LE'
J(3)='NA'
J(4)=':1'
      .
      .
      .
OPEN (I,J,K)
CLOSE (I)
```

EXAMPLE 8:

```
DIMENSION J(7)
DATA J/'FI','L:','1'/

OPEN (I,F,K)
CLOSE (I)
```

In Examples 7 and 8, the OPEN call will reslt in the default suffix (SA) being used. The logical drive for these two examples

is 1.

EXAMPLE 9:

```
DIMENSION J(7)
J(1)='FL'
J(2)='NA'
J(3)='ME'
J(4)='.F'
J(5)='D:'
J(6)='1'
      .
      .
      .
OPEN (I,J,K)
CLOSE (I)
```

EXAMPLE 10:

```
DIMENSION A(4)
DATA J/'FILENAME.SA:0'/
      .
      .
      .
OPEN (I,A,K)
CLOSE (I)
```

The file name, file suffix, and logical drive number are provided in Examples 9 and 10.

5.11 UNFORMATTED I/O

This version of MDOS FORTRAN allows unformatted READ and WRITE of data to and from disk files or external devices. The data will be written to the file in the same format that it is stored internally by MDOS FORTRAN.

Since the MDOS functions normally used on disk file I/O treat certain ASCII control characters as special, it is necessary to create and use a binary type file format, rather than ASCII. Therefore, to create and read binary files, these files are specified by different mode numbers. Specifically, the mode number 9 is used to read binary files, mode number 10 is used to create and write to a binary file, and mode number 11 is used to append to a binary file.

Other than use of different mode numbers associated with the OPEN statement, the only other difference is the omission of the format statement number in any READ or WRITE statement.

5.12 NON-SPACE COMPRESSED ASCII FILES

For certain applications, it may be desirable to create a disk file without the normal MDOS space compression. Use mode numbers

5, 6, and 7 in the OPEN statement to create and read this type file. These number correspond to 1, 2, and 3 respectively in normal modes as described in paragraph 5.10.1.

CHAPTER 6
DATA STATEMENT

GENERAL FORM

```
DATA k1/d1/,k2/d2,...../,kn/dn/
```

Where: Each k is a variable or array name. Dummy arguments may not appear in the list.
Each d is a list of constants (integer, real, or literal).

Literal data must be enclosed in apostrophes and may contain strings longer than a single storage element (2 for integer and 4 for real).

A DATA initialization statement is used to define initial values of variables, and arrays. There must be a one-to-one correspondence between the total number of elements specified or implied by the list k and the total number of constants specified by the corresponding list d.

This statement cannot precede any other specification statement that refers to the same variables or arrays. Otherwise, a DATA statement can appear anywhere in the program, but must not include variables declared to be in COMMON.

EXAMPLE

```
DIMENSION X(3)

DATA I/5/,J/-3/,X/8.0,-3.6,12.3/,N/'SA'/
```

The DATA statement indicates that the variables I, and J are to be initialized to the values 5, and -3, respectively. In addition, the statement specifies that the three variables in the array X are to be initialized to the values 8.0, -3.6, and 12.3. The integer variable N would contain the literal SA.

EXAMPLE

```
DIMENSION B(4)

DATA B/'FILENAME.SA:2'/
```

In this example, B(1) will contain the ASCII characters "FILE", B(2) contains "NAME", B(3) contains ".SA:", and B(4) will contain the "2" and be spaced filled for the other 3 bytes.

NOTE: Use of the DATA statement with the "R" compiler option

generates executable instructions which move the data from PSCT to the DSCT at execution time.

CHAPTER 7
SPECIFICATION STATEMENTS

7.1 INTRODUCTION

The specification statements provide the compiler with informatio about the nature of the data used in the source program. I addition, they supply the information required to allocat locations in memory for this data.

Specification statements must precede the program part containin executable statements. Within the specification statements, an statement describing data must precede referentes to that data The data must be defined before it is used.

7.2 DIMENSION STATEMENT

GENERAL FORM

```
DIMENSION a1(k1),a2(k2),a3(k3),....,an(kn)
```

where: a1, a2, a3, ... an are array names.
k1, k2, k3, ... kn are each composed of on through three unsigned integer constants separated by commas, representing the maximu value of each subscript in the array. Th maximum size of an integer constant is 32767 However, an array of this size would exce the available memory.

The information necessary to allocate storage for arrays used i the source program may be provided by the DIMENSION statement. The following example illustrates how this information may b declared.

EXAMPLE

```
DIMENSION A(10),ARRAY(5,5,5),LIST(10,100)

DIMENSION B(25,50),TABLE(5,8,4)
```

7.3 COMMON STATEMENT

GENERAL FORM

```
COMMON    a(k1),b(k2),c(k3),.....,an(kn)
```

Where: a,b,c... are variable names or array names that cannot be dummy arguments.
k1,k2,k3,...kn are required only if the variable represents an array name and are

each composed of one through
unsigned integer constants, separat
commas, representing the maximum val
each subscript in the array.

The COMMON statement is used to define a storage area that c
referred to by a calling program and one or more subprograms.
to specify the names of variables and arrays to be placed in
area. Therefore, variables or arrays that appear in a cal
program or subprogram can be made to share the same sto
locations with variables or arrays in other subprograms. Also
COMMON area can be used to implictly transfer arguments betwe
calling program and a subprogram. Arguments passed in COMMON
subject to the same rules with regard to type, length, etc.
arguments passed in an argument list (see Chapter 8, "PRO
UNITS").

IF more than one COMMON statement appears in a calling progra
subprogram, the entries in the statements are cumulat
Redundant entries are not permitted.

Since the entries in a COMMON area share storage locations,
order in which they are entered is significant. Consider
following example:

EXAMPLE

Calling Program	Subprogram
	SUBROUTINE MAPMY(. . .)
COMMON A, B, C, K(100)	
	COMMON X, Y, Z, J(100)
CALL MAPMY(. . .)	

In the calling program, the statement COMMON A, B, C, K(100) wo
cause 212 storage locations to be reserved in the following orde

BYTES	VARIABLE
4	A
4	B
4	C
2	K(1)
196	K(2) - K(99)
2	K(100)

The statement COMMON X, Y, Z, J(100) would then cause t variables X, Y, Z and J(1)...J(100) to share the same stora space as A, B, C, and K(1)...K(100), respectively. Note th values for X, Y, Z, and J(1) J(100), because they occupy t same storage locations as A, B, C, and K(1)...K(199), do not ha to be transmitted in the argument list of a CALL statement.

The use of a second COMMON in the calling program, preceding t existing COMMON, would cause the above example to be incorrect.

7.3 EQUIVALENCE STATEMENT

The EQUIVALENCE statement is used to define one or more variab name(s) equivalent to another variable. The same memory stora locations will be shared by one or more variable names.

The main use of this statement would be to save on memory si needed for a particular application.

GENERAL FORM:

EQUIVALENCE (a,b),(c,d)...,(x,y)

where each pair enclosed by parethesis are declare equivalent.

If either or both of the variables are dimensioned, they must hav been declared prior to using in an equivalence statement.

Example: Suppose there were two arrays in a program, A and B Let's dimension them first...

```
DIMENSION A(5),B(10)
```

Now, to make them occupy the same area in memory.

```
EQUIVALENCE (B,A)
```

or to make the 2nd element of A occupy the same memory locat
the 5th element of B...

```
EQUIVALENCE (B(5),A(2))
```

Note that reversing the two elements in the above statement
be illegal since it would cause the lowest 3 elements of arr
to fall lower than the start of array A.

7.4 EXTERNAL STATEMENT

This statement is used to declare a name to be an exte
reference rather than a variable name or subprogram name
program unit.

GENERAL FORM:

```
EXTERNAL n1,n2,...nN
```

where n1, n2, etc. are legal FORTRAN names.

After declaring external, the same name may not be used in
other way within the program unit. There are only cert
statements that this name may be used with...namely OPEN and CA
and then only as arguments.

CHAPTER 8
PROGRAM UNITS

8.1 INTRODUCTION

A program unit consists of a sequence of statements and may be either a main program or a subprogram. There may be only one main program unit, but several or no subprograms.

It is sometimes desirable to write a program which, at various points, requires the same computation to be performed with different data for each calculation. It would simplify the writing of that program if the statements required to perform the desired computation could be written only once and then could be referred to freely, with each subsequent reference having the same effect as though these instructions were written at the point in the program where the reference was made.

For example, to take the cube root of a number, a program must be written with this object in mind. If a general program were written to take the cube root of any number, it would be desirable to be able to combine that program (or subprogram) with other programs where cube root calculations are required.

The FORTRAN language provides for the above situation through the use of subprograms. There are two classes of subprograms: FUNCTION and SUBROUTINE. Functions differ from SUBROUTINE subprograms in that they return one value to the calling program, whereas SUBROUTINE subprograms need not return any.

8.2 NAMING PROGRAM UNITS

A program unit name consists of from one through six alphameric characters, the first of which must be alphabetic. A program unit name may not contain special characters (see Appendix A) or be a keyword (see Section 2.10). The type of a function name determines the type of the result that can be returned from it by the predefined convention for variable names.

The type of a SUBROUTINE subprogram cannot be defined because the results that are returned to the calling program are dependent only on the type of the variable names appearing in the argument list of the calling program and/or the implicit arguments in COMMON.

8.3 MAIN PROGRAM UNIT

A main program unit written in FORTRAN may call other subprogram units but not vice-versa. MDOS FORTRAN produces a main program unit when the first non-comment statement is not a SUBROUTINE, FUNCTION, or TASK statement. The main program unit will initialize the microprocessor's stack pointer (SP in the 6800, and both S and U in the 6809) before executing any other code.

8.3.1 PROGRAM Statement

The PROGRAM statement is optional in a main program. If used, it will become the module externally defined name in place of a default "MAIN@", and may be observed on the memory map produced by the linking loader (RLOAD).

The general form is:

PROGRAM name

where name is an acceptable name as defined in 8.2.

8.3.2 RETURN Statement

The RETURN statement is NOT permitted in a main program unit except in the case of a Real-Time FORTRAN main program unit which has called the subroutine SETRT.

8.4 FUNCTIONS

A function is a statement of the relationship between a number of variables. To use a function in FORTRAN, it is necessary to.

1. Define the function (i.e., specify which calculations are to be performed)

2. Refer to the function by name where required in the program.

3. A maximum of 13 arguments is permitted. Expressions are not permitted.

8.4.1 Function Definition

There are three steps in the definition of a function in FORTRAN:

1. The function must be assigned a unique name by which it may be called (see Paragraph 8.2).

2. The dummy arguments of the function must be stated.

3. The procedure for evaluating the function must be stated.

8.4.2 Function Reference

When the name of a function, followed by a list of its arguments, appears in any FORTRAN expression, it references the function and causes the computations to be performed as indicated by the function definition. The resulting quantity replaces the function reference in the expression, and assumes the type of the function. The type of the name used for the reference must agree with the type of the name used in the definition.

8.5 FUNCTION SUBPROGRAMS

The FUNCTION subprogram is a FORTRAN subprogram consisting of a FUNCTION statement followed by other statements, including at least one RETURN statement. It is an independently written program that is executed wherever ~~its~~ name is executed wherever its name is referenced in another program.

8.5.1 FUNCTION Statement

GENERAL FORM

FUNCTION name(a1,a2,a3,...an)

Where: name is the name of the FUNCTION.
a1,a2,a3, an are dummy arguments. They must be nonsubscripted variable, array, or dummy names of SUBROUTINE or other FUNCTION subprograms. (There must be at least one arguments in the argument list, and not more than 6).

Since the FUNCTION is a separate subprogram, the variables and statement numbers within it do not relate to any other program.

The FUNCTION statement must be the first statement in the subprogram. The FUNCTION subprogram may contain any FORTRAN statement except a SUBROUTINE statement or another FUNCTION statement.

The name of the function must be assigned a value at least once in the subprogram -- either as the variable name to the left of the equal sign in an assignment statement, as an argument of a CALL statement, or in an input list (READ statement) within the subprogram.

The dummy arguments of the FUNCTION subprogram (i.e., a , a , a , ...) may be considered to be dummy variable names. These are replaced at the time of execution by the actual argments supplied in the function reference in the calling program. Additional information about arguments is in Paragraph 8.7, "Arguments in a FUNCTION or SUBROUTINE Subprogram".

The relationship between variable names used as arguments in the calling program and the dummy variables used as arguments in the FUNCTION subprogram is illustrated in the following examples:

EXAMPLES

Calling Program	FUNCTION Subprogram
	FUNCTION CALC(A,B,J)

```
                                        I = J*2
ANS = ROOT1*CALC(X,Y,I
                                        CALC = A**I/B
                                        RETURN
                                        END
```

In this example, the values of X, Y, and I are used in the FUNCTION subprogram as the values of A, B, and J, respectively. The value of CALC is computed, and this value is returned to the calling program, where the value of ANS is computed. The variable I in the argument list of CALC in the calling program is not the same as the variable I appearing in the subprogram.

Calling Program

```
ANS = ROOT1*KALC(L, M, I)
```

FUNCTION Subprogram

```
FUNCTION KALC(I, J, K)
I = J*2
KALC = I+J+K**2
RETURN
END
```

The FUNCTION subprogram KALC is considered an INTEGER of length 2.

8.5.2 RETURN Statement

FUNCTION subprograms may contain a RETURN statement, which signifies a logical conclusion of the computation and returns the computed value and control to the calling program. There may be more than one RETURN statement in a FORTRAN subprogram or the RETURN statement may be omitted (the END statement in this case generates the return).

EXAMPLE

```
      FUNCTION DAV (D,E,F)
      IF (D-E) 10, 20, 30
10 A = D+2.0*F
```

```
      RETURN
30 DAV = D**2
      .
      .
      .
      RETURN
```

8.6 SUBROUTINE SUBPROGRAMS

The SUBROUTINE subprogram is similar to the FUNCTION subprogram in many respects. They both require an END and RETURN statement, and they both contain the same sort of dummy arguments. Like the FUNCTION subprogram, the SUBROUTINE program is a set of commonly used computations, but it need not return any results to the calling program, as does the FUNCTION subprogram. A maximum of 10 arguments is permitted. A space is not allowed between the subroutine name and the beginning of the argument list.

The SUBROUTINE subprogram is referenced by the CALL statement, which consists of the word CALL followed by the name of the subprogram and its parenthesized arguments.

8.6.1 SUBROUTINE Statement

GENERAL FORM

SUBROUTINE name(a1,a2,a3, . . . ,an)

Where: name is the SUBROUTINE name (see Paragraph 8.2, "Naming Subprograms").

a1,a2,a3, ... an are dummy input and/or output arguments. (There need not be any and maximum is 13)
Each argument used must be a nonsubscripted variable or array name.

Since the SUBROUTINE is a separate program, the variables and statement numbers within it do not relate to any other program.

The SUBROUTINE statement must be the first statement in the subprogram. The SUBROUTINE subprogram may contain any FORTRAN statement except a FUNCTION statement, another SUBROUTINE statement, or a PROGRAM statement.

The SUBROUTINE subprogram may use one or more of its arguments to return values to the calling program. Any arguments so used must appear to the left of an arithmetic statement, in an input list within the subprogram, as arguments of a CALL statement, or as arguments in a function reference. The keyword SUBROUTINE and the subroutine name must not appear in any other statement in the SUBROUTINE subprogram.

The dummy arguments (a1,a2,a3,...,an) may be considered dummy variable names that are replaced at the time of execution by the actual arguments supplied in the CALL statement. Additional information about dummy arguments is in Paragraph 8.7, "Arguments in a FUNCTION or SUBROUTINE Subprogram".

EXAMPLE

The relationship between variable names used as arguments in the calling program and the dummy variable used as arguments in the SUBROUTINE subprogram is illustrated in the following example. The object of the subprogram is to "copy" one array directly into another.

```
CALLING PROGRAM                 SUBROUTINE SUBPROGRAM
---------------                 ---------------------
DIMENSION X(100),Y(100)
                                SUBROUTINE COPY (A,B,N)
                                DIMENSION A (100),B(100)
K = 100                         DO 10 I = 1, N
CALL COPY (X,Y,K)     10        B(I) = A(I)
                                RETURN
                                END
```

8.6.2 CALL Statement

The CALL statement is used to call a SUBROUTINE program.

GENERAL FORM

```
CALL name(a1,a2,a3,...,an)
```

Where: name is the name of a SUBROUTINE subprogram. a1, a2, a3, ... an are the actual arguments that are being supplied to the SUBROUTINE subprogram. (Maximum of 13)

EXAMPLE

```
CALL OUT
CALL MATMPY(X,Y,Z,ROOT1,ROOT2)
```

The CALL statement transfers control to the SUBROUTINE program.

and replaces the dummy variables with the value of the actua
argments that appear in the CALL statement.

8.6.3' RETURN Statement

SUBROUTINE subprograms may contain a RETURN statement, whic
signifies a logical conclusion of the computation and return
control to the calling program. There may be more than one RETUR
statement in a SUBROUTINE subprogram or the RETURN statement ma
be omitted (the END statement in this case generates the return).

GENERAL FORM

```
RETURN
```

The normal sequence of execution following the RETURN statement o
a SUBROUTINE subprogram is to the next statement following th
CALL in the calling program.

8.7 ARGUMENTS IN SUBPROGRAMS

The dummy arguments of a subprogram appear after the FUNCTION o
SUBROUTINE name, and are enclosed in parentheses. They ar
replaced at the time of execution by the actual arguments supplie
in the CALL statement or function reference in the callin
program. The dummy arguments must correspond in number, order,
type, and length to the actual arguments. For example, if a
actual argument is an integer constant, then the correspondin
dummy argument must be an integer. If a dummy argument is a
array, the corresponding actual argument must be (1) an array, o
(2) an array element. In the first instance, the size of the
dummy array must not exceed the size of the actual array. In the
second, the size of the dummy array must not exceed the size o
that portion of the actual array which follows and includes the
designated element.

Following is an example of the actual argument being an array
element:

```
CALLING PROGRAM          SUBROUTINE SUBPROGRAM
---------------          ---------------------
DIMENSION A(6)           SUBROUTINE SAM(B)
     .                   DIMENSION B(2)
     .                        .
CALL SAM(A(5))                .
     .                   RETURN
     .                   END
```

In the above example, the portion of the actual array that follows
and includes the designated element is element 5 and element 6.
Therefore, dummy array B must not be larger than two.

The actual arguments can be:

Any type of constant
Any type of subscripted or nonsubscripted variable
An array name

If a literal constant is passed as an argument, the actual argument passed is the literal as defined, without delimiting apostrophes. A maximum of two characters can be passed as a literal.

When the dummy argument is an array name, an appropriate DIMENSION statement must appear in the subprogram. None of the dummy arguments may appear in a COMMON statement.

If a dummy argument is assigned a value in the subprogram, the corresponding actual argument must be a subscripted or unsubscripted variable name, or an array name. A constant should not be specified as an actual argument unless the programmmer is certain that the corresponding dummy argument is not assigned a value in the subprogram.

CHAPTER 9
6800 REAL TIME FORTRAN

9.1 INTRODUCTION

The Real-Time features available in the MDOS REAL TIME FORTRA version give the user the capability of writing real-time softwar in a high-level language for ultimate use in an EXORciser EXORterm, Micromodule or equivalent 6800 based system.

The Real-Time version not only is a language compiler, but als contains an execution-time operating system with several queues o tasks to be performed, along with an ability to respond t real-time interrupts and generation of delays.

9.2 REAL-TIME OPERATING SYSTEM

The operating system may be looked upon as having severa features, namely:

Task queues
Priorities
Interrupt handling
Delay queuing

9.2.1 Task Queues

There are five queues of tasks to be performed:

1. An active queue
2. A 10 millisecond timer queue
3. A 1 second timer queue
4. A 1 minute timer queue
5. An interrupt association queue

Tasks or segments of tasks which are to be executed afte specified time delays are placed in the 10 millisecond, 1 second, and 1 minute queues with associated counts of time delay units. The programmer can do this with calls to START and WAI subroutines as described later.

The operating system determines when tasks are to be transferre to the active queue based upon the specified time delays. Task with no time delays are entered in the active queue directly.

When a READ, PRINT, or WRITE statement is encountered, th operating system does not permit the system to be locked in th I/O operation as is the case in standard FORTRAN. The operatin system will start the next ready task in the active queue if delay is encountered in waiting for an I/O device to become ready. After the device has become ready, control will return to the I/ task.

9.2.2 Priorities

Associated with each task in the active queue is a priority lev
There are two classes of priorities: Immediate and Norma
Priority levels are numbered from 1 to 255. Immediate cl
priority levels are 1-127, while Normal class priority levels a
128-255. The lower the number, the greater the priority.

In either class, when a task is placed in the active queue wi
the same priority class as the currently executing task, the
current task will not immediately be suspended, regardless of
priority level. Instead, the newly invoked task must wait unt
the current task terminates or is delayed or performs standa
FORTRAN I/O. However, a task invoked with a priority in t
immediate class will cause a task with priority in the norm
class to be temporarily suspended until the task in the immedia
class has completed execution. A task with a normal prior
cannot cause the suspension of a task with immediate priority.

It is suggested that immediate class priorities only be used fo
short tasks requiring very high priority, since they actua
interrupt the execution of a normal priority task and da
integrity may be lost if data is common to both tasks.

9.2.3 Interrupt Handling

A special form of a subroutine subprogram, called a TASK, i
written to perform the desired operations upon receiving a
interrupt from some external device in the system.

The association between a particular interrupt and a named TASK
made with a subroutine CALL to ATTACH. Arguments passed with
call include the TASK name, the memory address of the interrupt
peripheral device, a mask to determine source of the interrupt o
type of device, and the priority level number of the TASK.

A given TASK can be attached to handle more than one periphe
device. The real-time operating system will prevent the same ta
from executing simultaneously for two or more interrupts.

9.2.4 Delay Queuing

Tasks can be invoked in either of two manners. One by extern
interrupt as depicted above, the other by placing it into a
queue by a subroutine call to START or STARTV.

The call to START (and STARTV) requires specification of the TA
name and an associated delay. The task is placed in an
appropriate timing queue (or into the active queue in case of no
delay). STARTV allows passing of two additional arguments, one o
them being a priority. START uses the current executing tas
priority level for the priority of the newly invoked task.

Delay control routines enable the currently executing task to b

suspended from execution for a period of time or until some eve occurs. This suspension allows other tasks to be executed. T subroutine names for this feature are WAIT and WAITE, whereby t first specifies a time value, and the second specifies an argume which must reach a value of zero before control is returned.

9.3 INVOKING REAL-TIME FEATURES

A call to a subroutine in the REAL-TIME FORTRAN library necessary to set up certain parameters, storage areas, and lin to the Real-Time operating system.

9.3.1 SUBROUTINE SETRT

This subroutine is used to initialize the Real-Time system. must be the first executable statement in a real-time program.

General Form:

CALL SETRT(a,c,m,p)

Where:
- a is the memory address of the console c control ACIA
- c is the memory address of the PTM (programmabl timer module, MC6840) or PIA (periphera interface adapter, MC6820/MC6821) used for th real-time clock.
- m is the mask for PTM ($01) or PIA ($80 or $40 clock interrupt bit.
- p is the name of an array used for queue dat pool.

An example of this would be a system containing an ACIA at $FCF4, a Micromodule MM01D board with PTM at address $EC18, and the arra named KPOOL is used for a data pool. The following would be th proper call to SETRT:

```
CALL SETRT($FCF4,$EC18,1,KPOOL)
```

9.3.2 QUEUE ARRAY

The queue data pool array is used to provide space for dynamic storage requirements of the real-time operating system. Storage entries are 10 bytes long (5 integer variables elements of an array). Queue entries are used for:

1. ATTACH calls.
2. START calls.
3. WAIT calls.
4. Outstanding TASKs awaiting completion.

The statement to allocate the necessary storage could be:

```
DIMENSION KPOOL(5,20)
```

if it were determined that 20 entries would be the maximum ever used at one time by the system. If the system required more than

this number, an execution time error would be issued to console terminal (if any).

9.4 TASK SUBPROGRAMS

A TASK subprogram is similar to a regular FORTRAN subrou written to handle interrupts. However, a TASK cannot be CAL like a SUBROUTINE, it can only be invoked by placing in the act queue by either a START (or STARTV) or using ATTACH and receiv an interrupt.

Except for the use of COMMON, only one byte of data can be pas to a TASK, and then only through the queue.

General Form:

TASK <name>

or

TASK <name>(p)

Where: <name> specifies the name of the task.

p specifies an optional parameter.

This statement must be the first statement in a task subprogr and is similar to a SUBROUTINE statement. Each task thus defi acts as an independent program, and may include any valid FORTI statement except PROGRAM, SUBROUTINE, or FUNCTION. It may h one or more RETURN statements, or none at all. It must end w an END statement at the physical end of its source statements.

Any TASK may invoke other tasks, call upon subroutines functions, and use COMMON in the normal manner.

9.5 START SUBROUTINE

The call to START is one of the methods of invoking a task.

General Form:

CALL START(<name>,i,j,k)

Where: <name> specifies the task name previously declared EXTERNAL.

specifies the minimum amount of time to delay before executing the specified task. Negative o zero values indicate no delay is required prior to execution.

specifies the value of the time unit associated with argument i as follows:

0 - unit of real-time clock (10 ms.)

1 10 milliseconds

2 seconds

3 - minutes

k specifies a return argument which indicates if the task was accepted, where:

1 - specifies accepted

2 or greater - specifies not accepted

The execution of this call has the effect of queuing the invok task with the same priority as the current task. Control maintained by the current task.

9.6 STARTV SUBROUTINE

A slight variation of the START subroutine permits a sing argument to be passed to the invoked task as well as allowing t task to be invoked with a different priority from the current executing task.
General Form:

CALL STARTV(<name>,i,j,k,arg,pri)

Where: <name> specifies the task name previously declared EXTERNAL.

i specifies the minimum amount of time to delay before executing the specified task. Negative or zero values indicate no delay is required prior to execution.

J specifies the value of the time unit associated with argument as follows:

0 unit of real-time clock (10 ms.)

1 - 10 milliseconds

2 seconds

3 - minutes

k specifies a return argument which indicates if the task was accepted, where:

1 - specifies accepted

2 or greater - specifies not accepted

arg specifies the task argument (integer only).

pri specifies the priority level of the invoked task.

The execution of this call has the effect of queuing the invoked task with the specified priority level. Control is maintained by the current task.

9.7 ATTACH SUBROUTINE

This subroutine sets up an association between an interrupt and the task which is to handle it.
General Form:

```
CALL ATTACH(<name>,addr,nnn,pri)
```

Where: <name> specifies the task to be associated with the interrupt, the name previously declared EXTERNAL

addr specifies the memory address of the peripheral device.

nnn specifies the interrupt mask bit of the peripheral device in cases of a PIA or PTM, or the driver address for other devices handled through drivers. RESTRICTION: Driver address must be above 00FF hexadecimal. See Chapter 10.

pri specifies the priority level of the task.

With this subroutine, a given task can be assigned to handle multiple peripheral devices. The real-time operating system will prevent the same task from executing simultaneously for two or more interrupts. The subroutine makes an entry in the interrupt queue upon execution. The interrupt queue is searched when an interrupt is received.

9.8 WAIT SUBROUTINE

This subroutine enables the currently executing task to be suspended from execution for a period of time, allowing other tasks to be executed.
General Form:

```
CALL WAIT(i,j,k)
```

Where: specifies the minimum amount of time to delay before resuming this task. Negative or zero values indicate no delay is required prior to resumption.

j specifies the value of the time unit associated with argument i as follows:

0 unit of real-time clock (10 ms.)

1 10 milliseconds

2 - seconds

3 - minutes

k specifies a return argument which indicates if the task was accepted, where:

1 - specifies accepted

2 or greater - specifies not accepted

9.9 WAITE SUBROUTINE

This subroutine suspends the currently executing task until a given event happens. Control will not be returned to the task until the variable has a value of zero. This suspension allows other tasks to be executed in the meantime.
General Form:

```
CALL WAITE(arg)
```

Where: arg specifies an integer variable with a value from 0 to 255. (Only the least significant byte is used.)

9.10 OTHER REAL-TIME SUPPORT SUBROUTINES

Some other subroutines which the real-time programmer may find useful are described in this section.

9.10.1 QCLEAR

This subroutine facilitates possible execution time error recovery if the data pool array containing the queues becomes overloaded. Execution of the subroutine essentially clears all real-time queues, enabling the programmer to start over. No arguments are permitted.

9.10.2 Single Byte I/O

Two subroutines enable the programmer to directly read and write single bytes to or from memory. This allows initialization of peripherial devices and has many other uses to the advanced programmer who must manipulate data in memory at the byte level.
General Forms:

```
CALL BI(adr,var)
CALL BO(adr,arg)
```

Where: adr specifies the memory address

var specifies the variable to receive the data

arg specifies a constant or variable containing data

The subroutine BI is used to input one byte (Byte In), while BO is used to output one byte (Byte Out). All variables and constants must be integer.

9.10.3 Double Byte I/O

Two subroutines enable the programmer to directly read and write double bytes to or from memory.
General Forms:

```
CALL DBI(adr,var)
CALL DBO(adr,arg)
```

Where: adr specifies the memory address

var specifies the variable to receive the data

arg specifies a constant or variable containing data

The subroutine DBI is used to input two bytes (Double Byte In), while DBO is used to output two bytes (Double Byte Out). All variables and constants must be integer.

9.10.4 Bit Manipulation

The function of bit manipulation operations is to provide an efficient means of packing and testing data. This is particularly useful in microprocessor programs, especially in relation to I/O handling. These routines are implemented as functions.

Function Name	Operation
IOR(j,m)	Performs logical inclusive OR.
IAND(j,m)	Performs logical AND (product).
INOT(j)	Performs logical complement.
IEOR(j,m)	Performs logical exclusive OR.
ISHFT(j,m)	Performs logical shift.
IBTEST(j,m)	Performs test of specific bit.
IBSET(j,m)	Performs setting of specifit bit.
IBCLR(j,m)	Performs clearing of specific bit.

These and other functions are described in more detail in Appendix D.

9.11 REAL-TIME PROGRAMMING HINTS

The methods used and the philosphy behind real-time systems are so different from conventional FORTRAN programming that it is appropriate to cover certain essential points. This is especially

necessary because of timing considerations and the interactions between the programmer's code and the operating system.

9.11.1 Use of the RETURN Statement

At least one RETURN (or END) statement must be used in every SUBROUTINE, FUNCTION, or TASK. This statement may also be used in a main real-time program if the main program does not contain instructions to be executed after execution of the initialization tasks.

Usually the beginning of the main program will perform task initialization. After initialization, there may be instructions to be executed in background or there may not. If there is, then the background code will consist of an endless loop. If there is no background code, then the RETURN statement is used. The reason for this non-standard usage is that any main program of a real-time system is itself considered to be a subroutine of the operating system.

9.11.2 Multiple Interrupts

External interrupts are handled by the TASK subprogram feature. Such interrupts may occur at any time and on occasion may follow one another very rapidly. It may, therefore, happen that while one external interrupt is being handled by its associated task, another external interrupt may occur. If this happens, the second interrupt will be placed in the active queue by the operating system.

Now, if the executing task contains a call to WAIT, WAITE, or an I/O statement, this will cause the operating system to suspend its operation and return to processing the active queue.

The operating system will not permit the task to be re-entered until it has completed execution on behalf of the first interrupt. This lockout provision thus prevents execution confusion and allows several such interrupts to be queued for execution in an orderly manner.

9.11.3 Data Read at Interrupt

When an external interrupt has been sensed, data will always be read from the corresponding PIA register or handled by the device driver. Note that this will occur at time of the interrupt, not at the time at which the task associated with the interrupt is executed. This data is available to the task if an argument was used in the TASK statement.

9.11.4 Task Sharing Same Subroutines

Two tasks may call the same standard subroutine if that subroutine does not contain a call to WAIT, WAITE, or an I/O statement. In order to understand the reason for this, consider the following

example:

```
      TASK A                          TASK B

      CALL XYZ                        CALL XYZ

      RETURN                          RETURN
      END                             END


                  SUBROUTINE XYZ

                  CALL WAIT(2,2,K)

                  RETURN
                  END
```

Task A is entered first and call subroutine XYZ. XYZ is then executed until the 2 second delay is encountered and control returns to the operating system. If another interrupt is now sensed which starts task B, subroutine XYZ will again be called and the system will fail.

A little consideration will show that this situation cannot occur if there is no WAIT, WAITE, or I/O within subroutine XYZ, since there will then be no possibility of returning to the operating system.

9.11.5 Processing Necessary Responses

The manner in which the operating system works, encourages the user to design interrupt tasks and subroutines which are relatively short. Lengthy computational routines may cause the system to be locked out of processing necessary responses to interrupts or other high frequency routines. A simple method of subdividing long computational routines is to use the statement: CALL WAIT(0,0,K) at appropriate intervals. This has the effect of passing control back to the operating system.

9.11.6 Task Stack Size Limitations

Each TASK subprogram has memory allocated for a stack area. The stack must handle return addresses for subroutine and function calls and at least 7 bytes for each interrupt received. A compromise size of 32 bytes is allocated by the compiler in DSCT. If the user finds this size is too small, he may easily increase this size by loading a simple module before loading each TASK module in which he desires the stack size allocation increased.

The assembly language source for this allocation module is shown below. It must be assembled as a relocatable module by the MDOS

Macro Assembler (RASM). It may be repeatedly loaded as needed.

```
NAM INCSTK
IDNT INCREASE DSCT STACK ALLOCATION BY $20
DSCT
RMB $20 (This value may be changed as desired)
END
```

9.12 END-SYSTEM HARDWARE CONSIDERATIONS

9.12.1 Real-Time Clock

A real-time clock is necessary for the operation of the REAL-TIME FORTRAN operating system. It has been determined that a 10 millisecond clock is a reasonable compromise in a microprocessor system between response time to interrupts, accuracy of timing, and overhead time associated with each "tick" of the clock.

Two methods are provided to implement this clock in hardware. One is to use a clock oscillator and bring it into the system via a PIA. The other is to use a PTM and generate the clock signal as a function of the MPU clock signal. MDOS REAL-TIME FORTRAN makes the assumption that if the PTM is used, the MPU clock frequency is 1.0 MHz. If other than this frequency, the routine in SETRT must be patched to take this into consideration.

9.12.2 No Console in System

A system can be devised that does not have any kind of operator's terminal or console I/O device. If this is the desired system, then several items must be considered.

First, what will happen if an execution time error occurs? There should be some provision to notify the operator of the end-system of this fact.

Second, it will be necessary to put a "dummy" address of 0000 for the ACIA in the CALL SETRT statement. This causes the operating system to ignore any initialization or attempt at I/O via the ACIA.

9.12.3 RAM at $0020-40

MDOS FORTRAN at present requires a small amount of RAM memory for use as scratchpad from $0020 to $0040. This memory must be present in the end-system.

9.12.4 MDOS Disk I/O

MDOS Disk I/O operation in a Real-Time system will cause the real-time clock interrupts (ticks) to be ignored during accesses to the disk. Therefore, any waits or other timing dependent upon the real-time clock would no longer be accurate or necessarily

consistant. When MDOS disk I/O is used, the PSCT portion of t program cannot be put into any type of ROM.

9.13 VECTORS FOR NMI, IRQ, AND RESTART

The following externally defined symbols may be used in defini the upper ROM vectors for the end-system:

IRQ$	IRQ vector (at $FFF8)
NMI$	NMI vector (at $FFFC)
START$	RES vector (at $FFFE)

The NMI in this Real-Time operating system saves the stack point in memory, sets a flag in another location, and then just goe into an endless loop.

The system may be restarted from the NMI condition by re-enterir at the location defined by the externally defined symbol "RESTR$" This entry point checks for the NMI flag, and if it finds i present will reload the stack pointer from the location it wa saved in previously. If it doesn't find the NMI flag, the stac is loaded from a predetermined location...the same as if th "START$" entry point was used. Therefore, "RESTR$" could be use in place of "START$" for the RES vector at $FFFE.

The NMI flag pattern used is $C3A5 and relies upon RAM memory no initializing to that exact two-byte pattern upon a power-up.

9.14 DEBUG OF REAL-TIME PROGRAMS

To assist the programmer in debugging Real-Time FORTRAN programs certain tools have been developed. This section gives certai information necessary to be able to find the cause of a syste crash or other malfunction.

It has been found to be almost a necessity to be able to disable temporarily the real-time clock interrupts from the system while certain debug procedures are used.

9.14.1 Queue Entry Formats

The following information is presented here to aid an advanced real-time programmer in determining malfuction causes:

QUEUE ENTRIES:

ACTIVE QUEUE (External reference link: AQ$)

	Used for
0-1	Link to next entry
2	Priority level
3	Stack Flag: 0=Entry mode/new stack, 1=Use old stack

4-5	Re-entry (task) or stack address
6-7	Data
8-9	Lock cell address

TIMER CONTROL (External reference link: TO$)

Bytes	Used for
(10 ms. queue)	
0-1	Link to first entry in queue array data pool
2	Initial value (1)
3	10 ms. Counter
(1 sec. queue)	
4-5	Link to first entry in queue array data pool
6	Initial value (100)
7	1 second counter
(1 min. queue)	
8-9	Link to first entry in queue array data pool
10	Initial value (60)
11	1 minute counter

TIMER QUEUE ENTRY FORMAT (first entry found from TIMER CONTROL)

Bytes	Used for
0-1	Link to next entry
2	Priority level
3	0=Entry mode, new stack, 1=Use old stack
4-5	Re-entry or stack address
6-7	Data
8-9	Counter

INTERRUPT QUEUE (External reference: IQ$)

Bytes	Used for
0-1	Link to next entry
2	Priority level
3	Bit to test (mask)/00=driver
4-5	Re-entry (task) address
6-7	Driver address or 0000
8-9	PIA/PTM address/device address

9.14.2 GDUMP Subroutine

This subroutine is supplied as part of the REAL-TIME FORTRAN run time library (FORLB.RO). It may be called at any time during a program and will use whatever FORTRAN console I/O is provided by the user. The purpose of the subroutine is to produce a dump of the various queues in an orderly fashion, either to the console or

A possible use of this subroutine might be to call it whenever an error or a particular error is observed. Error calls may be intercepted through the .ADDR jump table as outlined in Appendix I.

CHAPTER 10
EXTERNAL DEVICE DRIVERS

10.1 INTRODUCTION

MDOS REAL-TIME FORTRAN supports external device drivers in a way which makes I/O to devices other than console terminal, line printer, and MDOS disk quite easy and efficent.

Most device drivers will be written in assembly language and assembled as a relocatable module to be linked to FORTRAN when required. Normal FORTRAN statements such as OPEN, CLOSE, READ, and WRITE will be used by the programmer to access most any device.

10.2 FORTRAN I/O STATEMENTS

The MDOS FORTRAN statements which may be used by the programmer in implementing external devices for I/O are described in this context in the following paragraphs.

10.2.1 EXTERNAL

The EXTERNAL statement is used to declare the name given to the device driver as external to the program, distinguishing it from an internal variable or subprogram name.

Example: EXTERNAL ACIADV

where "ACIADV" is the name of the driver routine to be used.

10.2.2 OPEN

Once the driver routine has been declared EXTERNAL, it may be associated with a particular FORTRAN file reference number with an OPEN statement. This is very similar to opening a disk file, except that the external name of the driver routine is supplied in place of the disk file name, and the external device address is supplied in place of the file mode.

General Form:

```
OPEN (n,x,a)
```

Where: n specifies the file reference number

x specifies the external device driver routine name

a specifies the external device address

The same driver may be used for more than one device by additional OPEN statements referencing that driver.

Example: OPEN (7,ACIADV,$EC90)

This example causes the following action upon execution:

1. Associates unit number 7 with using a driver name "ACIADV" for a device physically located at memory address $EC90.

2. Goes to the driver routine "ACIADV" to perform any initialization of the device located at $EC90. For some devices, no initialization is necessary.

NOTE: Use of the OPEN statement in conjunction with an EXTERNAL routine overrides any previous assignment of a file reference number to either a disk file or a pre-assigned unit (such as 102 for line printer) until a corresponding CLOSE statement is issued, at which time, any previous assignment is restored.

10.2.3 READ/WRITE

After the external driver routine has been defined and a reference number assigned, normal READ and WRITE statements may be used in conjunction with FORMAT statements to perform device I/O through the driver routine.

```
Example:        WRITE (7,900)
          900   FORMAT (' ENTER NUMBER & NAME >> ')
                READ (7,901) INUMB,NAME1
          901   FORMAT (I3,7A2)
```

Notice that normal READ, WRITE, and FORMAT statements are used. This is true except where only certain bits or bytes are required to be changed on output. See the "B" format editing code for an explanation on how to do this.

10.2.4 CLOSE

Termination of the external device is done by a CLOSE statement.

Example: CLOSE (7)

This statement causes the following action upon execution:

1. The driver executes any desired termination routine for the device.

2. Unit 7 is dis-associated from the driver and device and is now available for re-use if desired.

10.3 SUPPORTING SUBROUTINES

There are two supporting subroutines available for use with an external driver. These are DEVON and DEVOFF.

During execution of a FORTRAN program, additional ON/OFF control may be optionally exercised in relation to the external device. Depending upon how this feature is implemented in the device driver, a CALL DEVON or CALL DEVOFF will take the intended action.

General Form: CALL DEVON(n)
CALL DEVOFF(n)

Where: n: specifies the file reference number assigned by an OPEN statement.

10.4 DRIVER STRUCTURE

All external device drivers used with MDOS FORTRAN must adhere t certain conventions. These are outlined in the followin paragraphs.

10.4.1 VECTOR TABLE

Each driver must have a vector table, the start of whic corresponds to the XDEF of the driver name. The vector entrie are described below:

Bytes	Pointer to Function	Called by
0-1	Initialize the device	OPEN
2-3	Terminate the device	CLOSE
4-5	Input to I/O buffer	READ
6-7	Output from I/O buffer	WRITE
8-9	Poll for IRQ	RTMOD routine
A-B	Reserved	
C-D	Turn on device	DEVON
E-F	Turn off device	DEVOFF

If any particular function is not implemented, the vector address given should point to an RTS instruction. All vector routine must end with an RTS.

The device address (if any) is passed to the driver through an externally defined symbolic address of DV$ADR except for IRQ handling where accumulators A and B are used. I/O is passed back and forth between FORTRAN and the driver through a buffer defined by the symbol BUF$.

On a WRITE statement in FORTRAN, one formatted line of output is placed in BUF$ buffer, then control is passed to the driver (through the vector at bytes 6-7). It is then the responsibility of the driver to take the data from the buffer and send it out to the external device.

On a READ statement in FORTRAN, control is passed to the driver (through the vector at bytes 4-5). It is the driver's responsibility to receive data from the external device, place it in the BUF$ buffer with an ASCII EOT ($04) character at the end, and then return control (via RTS) to FORTRAN to get the data from the buffer and place it in the variable list associated with the READ statement.

10.4.2 BUFFERS

Normally, most of the I/O will use only BUF$ as the buffer. However, in certain interrupt driven systems, it may be desirable for the device driver routine to have an additional buffer of its own. This allows the driver to transfer at high speed the contents of its own buffer to BUF$ or vice-versa, when needed, thus freeing up BUF$ for other I/O in the system.

An example of this might be when a system was writing to a line printer and inputting from the keyboard at the same time. Here, it would be advantageous for the keyboard input driver and line printer driver routines to each have their own buffer, using BUF$ only when needed by FORTRAN.

10.4.3 INTERRUPT HANDLING

Since interrupts may come from many different sources in a system, software polling must be done to find the source of the interrupt. Provision has been made through driver vector bytes 8-9 to allow polling of the external device for an interrupt. Accumulators A and B will contain the device address (A most significant byte of address). The driver must return accumulator A cleared if the device did not cause the interrupt, or accumulator A as non-zero if an interrupt is detected. In addition, any data to be returned upon detecting an interrupt must be passed in the index register by the driver.

Clearing of the interrupt source is accomplished through this driver routine before return to the caller.

10.4.4 Driver Address Restrictions

If the subroutine ATTACH is used, a device driver cannot start at any address below $0100. Normally, this is no restriction to be concerned with as most systems will use this area for either RAM or I/O devices, not program memory.

10.5 SAMPLE DRIVERS

The following is a source listing of a general purpose ACIA driver which may be modified by the user to suit the application. Interrupts are inhibited in this version. Assumption is made that the ACIA clock divide ratio is 16 and that 7 bits of data, 1 stop bit, and even parity is being used.

```
            NAM ACIADV
            XDEF ACIADV
            XREF BUF$,DV$ADR

            DSCT
    BPTR    RMB 2                   BUFFER POINTER STORAGE

            PSCT
    ACIADV  EQU *
            FDB DEVINT
            FDB DEVTRM
```

```
        FDB DEVIN
        FDB DEVOUT
        FDB DEVIP
        FDB DUMMY
        FDB DEVON
        FDB DEVOFF

* UNIMPLEMENTED VECTORS
DEVTRM EQU *
DEVIP  EQU *
DUMMY  EQU *
DEVON  EQU *
DEVOFF EQU *
       RTS

* INITIALIZATION OF ACIA
DEVINT LDX DV$ADR              GET ACIA ADDRESS
       LDAA #3
       STAA 0,X                MASTER RESET
       LDAA #%00001001
       STAA 0,X                INITIALIZE
       RTS

* INPUT FROM ACIA
DEVIN  LDX #BUF$+1
DEVIN2 STX BPTR
       LDX DV$ADR              GET ACIA ADDRESS
DEVIN4 LDAA 0,X                STATUS
       LSRA
       BCC DEVIN4              NOT READY
       LDAA 1,X                GET DATA
       CMPA #$0D               CR?
       BEQ DEVIN9              YES
       LDX BPTR                GET BUFFER POINTER
       STAA 0,X
       INX
       CPX #BUF$+132           END OF BUFFER YET?
       BNE DEVIN2
DEVIN9 LDAA #4                 EOT
       STAA 0,X                MARK END
       RTS

* OUTPUT TO ACIA
DEVOUT LDX #BUF$+1
DEVO2  STX BPTR
       LDAA 0,X                GET CHARACTER
       CMPA #4                 EOT?
       BEQ DEVO9
       BSR SEND
       LDX BPTR
       INX
       BRA DEVO2

DEVIN9 LDAA #$0D               CR
```

```
        BSR SEND
        LDAA #$0A           LF
        BSR SEND
        CLRA                NULL
        BSR SEND
        RTS

SEND    LDX DVSADR          GET ACIA ADDRESS
SEND2   LDAB 0,X            STATUS
        LSRB
        LSRB
        BCC SEND2           NOT READY
        STAA 1,X            SEND CHARACTER
        RTS
```

CHAPTER 11
INTERFACING WITH MICROMODULES

11.1 INTRODUCTION

Micromodules are a series of Motorola OEM boards, each with various features and functions. MDOS REAL-TIME FORTRAN has been written with these boards in mind and in most cases can be used in a system comprised of one or more Micromodules. This chapter describes the interfacing and use of some Micromodules with FORTRAN.

11.2 MICROMODULE 14/14A

Micromodule 14 (and 14A) allows use of an Arithmetic Processor Unit (APU) with 6800/6809 family microprocessors. The REAL-TIME version of MDOS FORTRAN may interface and use the APU on the Micromodule 14 and 14A boards to increase execution speed of real number arithmetic operations and allow several more trig functions than present in the FORTRAN library.

11.2.1 Using MM14 or MM14A

The programmer simply specifies use of a Micromodule 14 (or 14A) board in the end-system at compile time through the "M" option letter on the command line. See Chapter 1 concerning the command line of the compiler.

The rest is automatic, as during link time, the proper modules from the FORTRAN library will be searched for and loaded.

11.2.2 MM14/14A Precautions

The programmer must assure that ALL modules (outside the supplied FORTRAN library) have been compiled with the "M" option, otherwise there will be a symbol conflict during link time.

The FORTRAN program makes no checks to ascertain that a MM14 board is present in the system. If the board is not present in the system at execution time, rather unpredictable results will take place. Most likely, the program will either hang up in a loop or will abort due to an execution time error (usually an overflow error).

APPENDIX A
SOURCE PROGRAM CHARACTERS

Alphabetic Characters

A	N
B	O
C	P
D	Q
E	R
F	S
G	T
H	U
I	V
J	W
K	X
L	Y
M	Z

Numeric Characters

0	5
1	6
2	7
3	8
4	9

Special Characters

(blank)	*
+	, (comma)
-	(
/	' (apostrophe)
=	&
)	$
	. (period)

The 49 characters listed above constitute the set of characters acceptable by MDOS FORTRAN, except in literal data, where any valid ASCII character except the apostrophe is acceptable.

APPENDIX B
COMPILER ERROR MESSAGES

When errors are detected by the compiler, the message:

*** ERROR code

is printed at the console terminal where "code" represents one of the coded errors in the list below.

EXAMPLE:

```
0250        IF(J-3 10,20,30

*** ERROR 05
```

00 illegal character

01 non-numeric statement number

02 program contains too many variable names, symbol table overflow

03 statement is too complex for compiler

04 string is too long

05 syntax error

06 too many arguments (13 maximum)

07 numeric value too large

10 duplicate statement label

11 name already defined

12 array dimension too large

13 COMMON variables cannot be initialized in DATA statements

14 name too long (6 character maximum)

15 PROGRAM, SUBROUTINE, TASK, or FUNCTION statement not first

16 DATA variable does not match data type

17 subroutine name and variable name conflict

18 must be integer argument

20 too many statement labels with computed GOTO (20 max)

22 dummy argument name already used

23 too many external references

24 common or dummy argument not permitted

25 EQUIVALENCE not permitted

26 E and F editing codes not permitted with I option

30 over 10 operands in this statement

31 number of subscripts does not agree with number of dimensions

50 too many nested DO's (10 maximum)

51 one of the DO arguments is not an integer

53 END IF without matching IF-THEN

54 END IF missing

55 too many nested IF-THEN's (10 maximum)

56 branch out of range in logical IF

APPENDIX C
EXECUTION TIME ERROR MESSAGES

If a fatal error is detected during execution, the message:

```
***EXECUTION TIME ERROR #nn
    xxxx
    xxxx
    xxxx
    xxxx
```

will be displayed on the console terminal. In the above example, "nn" represents the error number and the four lines of "xxxx" represent the last 4 double bytes found on the stack (SP on 6800 and S on 6809). These values normally represent the subroutine return addresses and can be of some aid to locating what routines were called/executing when the error was encountered.

The following is a listing of the execution time error numbers and their meaning:

01	POWER function cannot be called with a -X
02	Cannot take log of a negative number
03	Cannot take SIN or COS of a negative number
04	Cannot take SQRT of a negative number
05	Only bit positions 0-15 valid
30	Call argument & dummies unequal in number
31	Integer formats only
40	Too many nested repeats in FORMAT
41	OPEN/CLOSE arguments must be integer
42	Invalid OPEN mode
43	Conflicting file modes
44	Attempt to access file not open
45	Maximum number of files already open
46	EOFTST or SETEOF with file not open
47	Attempt to REWIND file open for output
48	File number already open

49	Fatal MDOS related error
50	Subscript exceeds allowed range
	Integer overflow
52	Real overflow
53	Attempt to position LSN past EOF
54	Involution value not integer
55	Floating point routines missing
56	Cannot use BACKSP or DELR after REWIND or WRITE, or o files open for output. Cannot use SETLSN after a WRITE.
64	Attempt to access device not open
65	Max number of devices already open
68	Device number already open
71	Data pool variable not array
72	Data pool buffer overflow
73	Invalide real-time clock mask
74	Attempt to "CALL" a TASK subprogram
75	ACIA framing error

APPENDIX D
MDOS FORTRAN SUPPLIED FUNCTIONS

NAME	TYPE OF ARGUMENT	TYPE OF FUNCTION	NUMBER OF ARGUMENTS	DESCRIPTION
SQRT(X)	Real	Real	1	square root of X
EXP(X)	Real	Real	1	e**X
SIN(X)	Real	Real	1	sin of X
COS(X)	Real	Real	1	cosine of X
ATAN(X)	Real	Real	1	arctangent of X
ALOG(X)	Real	Real	1	natural logarithm of X
ABS(X)	Real	Real	1	absolute value of X
POWER(X,Y)	Real	Real	2	computes X**Y
MOD(I,J)	Integer	Integer	2	computes I modulo J
IAND(I,J)	Integer	Integer	2	logical AND of I and J
IOR(I,J)	Integer	Integer	2	logical OR of I and J
IEOR(I,J)	Integer	Integer	2	logical EOR of I and J
ISHFT(I,J)	Integer	Integer	2	shift I left or right J positions (+ left shift, - right shift)
IBTEST(I,J)	Integer	Integer	2	tests the Jth bit of I and returns 0 or 1.
IBCLR(I,J)	Integer	Integer	2	clears the Jth bit of I and returns the new value of I.
IBSET(I,J)	Integer	Integer	2	sets the Jth bit of I and returns the new value of I.

NOTE: Arguments must be a single variable, subscripted variable or constant. Expressions are not allowed. The type of arguments are important because improper types may return incorrect values from the function.

APPENDIX E
SUPPLIED SUBROUTINES

-to be supplied-

APPENDIX F
EXAMPLES OF FORTRAN

-to be supplied-

APPENDIX G

LINKING FORTRAN & ASSEMBLY LANGUAGE PROGRAMS

-to be supplied-

APPENDIX H
CREATING A LIBRARY OF ROUTINES

A library is created with the MERGE command. Consider how programs call one another before merging. For instance, if program A calls program B, program A must be merged first. Otherwise, the library must be searched twice. See the Linking Loader manual for more details.

For example, suppose you want to put a subroutine called PORT in FORLB.

```
=MERGE FORLB.RO,PORT.RO,MYLIB.RO
```

Program PORT now follows all of the FORTRAN library in a library called MYLIB.

APPENDIX I
CHANGING RUNTIME I/O ADDRESSES

It is sometimes desirable to execute FORTRAN programs on non-EXBUG systems. In order to do this, the EXBUG I/O addresses must be changed to those of the system where the program will be executed.

This may mean writing the necessary substitute routines to handle the I/O used. Listed below is a jump table that must be modified when using non-EXBUG I/O.

```
.ADDR COMM PSCT (.ADDR MUST BE USED)
 JMP $NNNN CONSOLE INPUT
 JMP $NNNN CONSOLE OUTPUT
 JMP $NNNN LINE PRINTER OUTPUT
 JMP $NNNN RETURN TO OPERATING SYSTEM
 JMP $NNNN ERROR ROUTINE
```

NOTE: $NNNN will be the address of your I/O routine for the particular operation. On input, the data must be put into the external reference BUF$ (a buffer). The data must be ASCII and must be terminated with an "EOT" (hexadecimal 04). On output, the data will be in BUF$ in ASCII, terminated with an "EOT".

Assemble into a relocatable object module and load, using the linking loader AFTER the library is loaded, or merge with your library (see Appendix H).

APPENDIX J

LINKING USING MDOS FORTRAN

```
=RLOAD

MDOS LINKING LOADER REV X.XX
COPYRIGHT BY MOTOROLA 1977

?IF=TEMP
?BASE
?LOAD=TEST
?LIB=FORLB
?OBJA=TEST
?EXIT
```

The use of the 'BASE' command is optional unless MDOS disk I/O used.

APPENDIX K
USING FORTRAN WITH READ-ONLY MEMORY

MDOS FORTRAN has been implemented such that the end user may pl his program in some form of Read Only Memory (ROM) and operate in a system other than a development EXORciser.

The requirements for a candidate ROM program to meet are:

1. The program does not use MDOS disk I/O.

2. of the "R" option during all FORTRAN mod compilations.

3. Any non-FORTRAN modules (such as assembly language) ROM-able.

4. User will have RAM in his system between $0020 and $0 for FORTRAN's unrestricted use.

The actual division of ROM/RAM comes about during link load t where the user must specify the start addresses of CSCT, DSCT, PSCT. The ROM-able portion of a FORTRAN program is PSCT. B CSCT (if used) and DSCT must be assigned to memory contain RAM.

If operated without EXbug (the EXORciser's monitor), then see section on implementing your own I/O.